



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ



Facultad de Ingeniería.
Área de Ciencias de la Computación.

Alumnos:

- Bryan Aarón Reyes Hernández 266902
- Mariana Sarahi Pérez Salinas 335987

Materia:

Administración de Bases de Datos.

Semestre:

2024-2025/I

Grupo:

231301

Profesor:

M.E.M. Claudia Alicia Méndez Hernández

Manual del Programador para la Base de Datos "Almacenamiento" y la interfaz del sistema en C#.

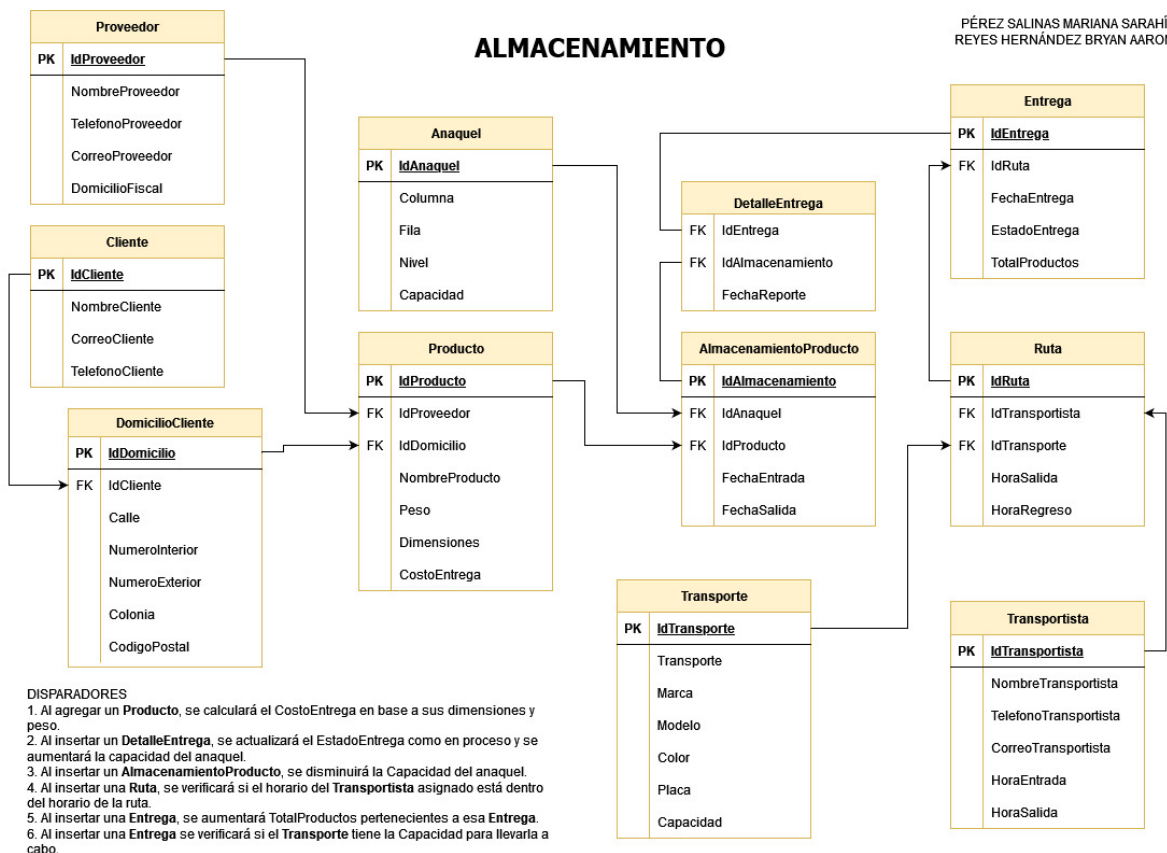
Introducción:

La base de datos, denominada "Almacenamiento", está implementada en SQL Server y se encarga de gestionar todas las operaciones relacionadas con la administración de inventarios y procesos logísticos. Su diseño incluye:

- Esquemas organizados para separar las áreas de almacenamiento y logística.
- Tablas optimizadas para almacenar información de productos, ubicaciones, movimientos y usuarios del sistema.
- Disparadores que garantizan la integridad de los datos y automatizan tareas específicas dentro del programa.

Este proyecto también implementa un sistema de gestión de información que utiliza C# como lenguaje de programación para el desarrollo de una interfaz que permite realizar operaciones de altas, consultas y eliminaciones de registros. El sistema está diseñado para manejar información relacionada con el almacenamiento y la logística de productos, proporcionando una solución eficiente y automatizada conectada a esta base de datos.

Diagrama de la base de datos:



Creación de Bases de datos.

Para crear la base de datos denominada "**Almacenamiento**", se debe utilizar el siguiente conjunto de instrucciones en **SQL Server**. Este procedimiento establece la base de datos y selecciona su contexto para permitir la creación de objetos dentro de ella.

```
CREATE DATABASE Almacenamiento
```

```
USE Almacenamiento
```

Esquemas principales

almacén:

Contiene tablas relacionadas con la gestión de productos, almacenamiento de productos, proveedores y anaqueles.

logística:

Contiene tablas para clientes, rutas, trasportes, domicilios de clientes, entregas y transportistas.

```
-- Esquemas
```

```
CREATE SCHEMA almacen
```

```
CREATE SCHEMA logistica
```

Tablas principales de la base de datos:

A continuación, se detallan las tablas principales con sus respectivos atributos de las entidades:

Tabla almacen.Proveedor

Esta tabla se encarga de almacenar los datos de los diferentes proveedores. Los datos que se registran son: nombre del proveedor, teléfono del proveedor, correo electrónico del proveedor y su domicilio fiscal. Esta tabla cuenta con un identificador único. Además, se incluye una restricción para que el correo y el teléfono del proveedor solo pueda ser único e irrepetible de los demás valores.

```
CREATE TABLE almacen.Proveedor
(
    IdProveedor BIGINT IDENTITY (1,1) NOT NULL,
    NombreProveedor VARCHAR(100) NOT NULL,
    TelefonoProveedor VARCHAR(10) NOT NULL,
    CorreoProveedor VARCHAR(100) NOT NULL,
    DomicilioFiscal VARCHAR(100) NOT NULL,

    CONSTRAINT PK_PROVEEDOR PRIMARY KEY (IdProveedor)
)

ALTER TABLE almacen.Proveedor
ADD CONSTRAINT UQ_CORREO_PROVEEDOR UNIQUE (CorreoProveedor),
    CONSTRAINT UQ_TELEFONO_PROVEEDOR UNIQUE (TelefonoProveedor);
```

Tabla logistica.Cliente:

Esta tabla se encarga de almacenar los datos de los clientes. Los datos registrados incluyen: nombre del cliente, correo electrónico del cliente y número telefónico del cliente. La tabla cuenta con un identificador único para cada cliente

```
CREATE TABLE logistica.Cliente
(
    IdCliente BIGINT IDENTITY (1,1) NOT NULL,
    NombreCliente VARCHAR(100) NOT NULL,
    CorreoCliente VARCHAR(100) NOT NULL,
    TelefonoCliente VARCHAR(10) NOT NULL,

    CONSTRAINT PK_CLIENTE PRIMARY KEY (IdCliente)
)
```

Tabla almacen.Anaquel:

Esta tabla se encarga de almacenar la información relacionada con los anaqueles utilizados para el almacenamiento. Los datos registrados incluyen: columna, fila, nivel y capacidad del anaquel. Cada registro cuenta con un identificador único.

```
CREATE TABLE almacen.Anaquel
(
    IdAnaquel BIGINT IDENTITY (1,1) NOT NULL,
    Columna INT NOT NULL,
    Fila INT NOT NULL,
    Nivel INT NOT NULL,
    Capacidad INT,

    CONSTRAINT PK_ANAQUEL PRIMARY KEY (IdAnaquel)
)
```

Tabla logistica.Transporte:

Esta tabla se encarga de almacenar la información de los medios de transporte utilizados en la logística. Los datos registrados incluyen: tipo de transporte, marca, modelo, color, placa y capacidad. La tabla cuenta con un identificador único para cada transporte. Además, se incluye una restricción para que el tipo de transporte solo pueda ser uno de los siguientes valores: 'Camión', 'Camioneta', 'Motocicleta' o 'Coche'.

```
CREATE TABLE logistica.Transporte
(
    IdTransporte BIGINT IDENTITY (1,1) NOT NULL,
    Transporte VARCHAR(100) NOT NULL,
    Marca VARCHAR(100) NOT NULL,
    Modelo VARCHAR(100) NOT NULL,
    Color VARCHAR(100) NOT NULL,
    Placa VARCHAR(100) NOT NULL,
    Capacidad INT,

    CONSTRAINT PK_TRANSPORTE PRIMARY KEY (IdTransporte),
    CONSTRAINT CHK_TRANSPORTE CHECK (Transporte IN ('Camión', 'Camioneta', 'Motocicleta', 'Coche'))
)
```

Tabla logistica.Transportista:

Esta tabla se encarga de almacenar la información de los transportistas. Los datos registrados incluyen: nombre del transportista, número telefónico, correo electrónico, hora de entrada y hora de salida. La tabla cuenta con un identificador único para cada transportista.

```
CREATE TABLE logistica.Transportista
(
    IdTransportista BIGINT IDENTITY (1,1) NOT NULL,
    NombreTransportista VARCHAR(100) NOT NULL,
    TelefonoTransportista VARCHAR(10) NOT NULL,
    CorreoTransportista VARCHAR(100) NOT NULL,
    HoraEntrada TIME NOT NULL,
    HoraSalida TIME NOT NULL,

    CONSTRAINT PK_TRANSPORTISTA PRIMARY KEY (IdTransportista)
)
```

Tablas secundarias con relaciones de la base de datos:

A continuación, se detallan las tablas que tienen relaciones con otras entidades, junto con sus respectivos atributos.

Tabla logistica.DomicilioCliente:

Esta tabla se encarga de almacenar los domicilios de los clientes. Los datos registrados incluyen: calle, número interior, número exterior, colonia y código postal. La tabla también incluye una referencia a la tabla logistica.Cliente a través de la columna IdCliente, estableciendo una relación entre los domicilios y los clientes. Esta tabla cuenta con un identificador único para cada domicilio.

```
CREATE TABLE logistica.DomicilioCliente
(
    IdDomicilio BIGINT IDENTITY (1,1) NOT NULL,
    IdCliente BIGINT NOT NULL,
    Calle VARCHAR(100) NOT NULL,
    NumeroInterior VARCHAR(100) ,
    NumeroExterior VARCHAR(100),
    Colonia VARCHAR(100) NOT NULL,
   CodigoPostal VARCHAR(10) NOT NULL,

    CONSTRAINT PK_DOMICILIO PRIMARY KEY (IdDomicilio),
    CONSTRAINT FK_CLIENTE FOREIGN KEY (IdCliente) REFERENCES logistica.Cliente(IdCliente)
)
```

Tabla almacen.Producto:

Esta tabla se encarga de almacenar los productos disponibles en el almacén. Los datos registrados incluyen: el nombre del producto, peso, dimensiones y el costo de entrega. Además, la tabla establece relaciones con la tabla almacen.Proveedor mediante el campo IdProveedor, y con la tabla almacen.Domicilio a través de la columna IdDomicilio. La tabla cuenta con un identificador único para cada producto

```
CREATE TABLE almacen.Producto
(
    IdProducto BIGINT IDENTITY(1,1) NOT NULL,
    IdProveedor BIGINT NOT NULL,
    IdDomicilio BIGINT NOT NULL,
    NombreProducto VARCHAR(100) NOT NULL,
    Peso FLOAT NOT NULL,
    Dimensiones VARCHAR(50) NOT NULL,
    CostoEntrega MONEY,

    CONSTRAINT PK_PRODUCTO PRIMARY KEY (IdProducto),
    CONSTRAINT FK_PROVEEDOR FOREIGN KEY (IdProveedor) REFERENCES almacen.Proveedor(IdProveedor)
)
```

Tabla almacen.AlmacenamientoProducto:

Esta tabla se encarga de registrar el almacenamiento de los productos en los anaqueles. Los datos registrados incluyen: el identificador del anaquel, el identificador del producto, la fecha de entrega y la fecha de salida del producto. La tabla establece relaciones con la tabla almacen.Anaquel a través de la columna IdAnaquel, y con la tabla almacen.Producto mediante la columna IdProducto. La tabla cuenta con un identificador único para cada registro de almacenamiento.

```
CREATE TABLE almacen.AlmacenamientoProducto
(
    IdAlmacenamiento BIGINT IDENTITY(1,1) NOT NULL,
    IdAnaquel BIGINT NOT NULL,
    IdProducto BIGINT NOT NULL,
    FechaEntrega DATE NOT NULL,
    FechaSalida DATE NOT NULL,

    CONSTRAINT PK_ALMACENAMIENTO PRIMARY KEY (IdAlmacenamiento),
    CONSTRAINT FK_ANAQUEL FOREIGN KEY (IdAnaquel) REFERENCES almacen.anaquel(IdAnaquel),
    CONSTRAINT FK_PRODUCTO FOREIGN KEY (IdProducto) REFERENCES almacen.Producto(IdProducto)
)
```

Tabla logistica.Ruta:

Esta tabla se encarga de almacenar la información relacionada con las rutas de transporte. Los datos registrados incluyen: el identificador del transportista, el identificador del transporte, la hora de salida y la hora de regreso. La tabla establece relaciones con la tabla logistica.Transportista a través de la columna IdTransportista, y con la tabla logistica.Transporte mediante la columna IdTransporte. La tabla cuenta con un identificador único para cada ruta.

```
CREATE TABLE logistica.Ruta
(
    IdRuta BIGINT IDENTITY(1,1) NOT NULL,
    IdTransportista BIGINT NOT NULL,
    IdTransporte BIGINT NOT NULL,
    HoraSalida TIME NOT NULL,
    HoraRegreso TIME NOT NULL,

    CONSTRAINT PK_RUTA PRIMARY KEY (IdRuta),
    CONSTRAINT FK_TRANSPORTISTA FOREIGN KEY (IdTransportista) REFERENCES
logistica.Transportista(IdTransportista),
    CONSTRAINT FK_TRANSPORTE FOREIGN KEY (IdTransporte) REFERENCES
logistica.Transporte(IdTransporte)
)
```

Tabla logistica.Entrega:

Esta tabla se encarga de almacenar los detalles de las entregas realizadas. Los datos registrados incluyen: el identificador de la ruta asociada, la fecha de entrega, el estado de la entrega y el total de productos entregados. La tabla establece una relación con la tabla logistica.Ruta mediante la columna IdRuta, y contiene una restricción que limita los valores posibles del estado de la entrega a 'No entregado', 'En proceso' o 'Entregado'. La tabla cuenta con un identificador único para cada entrega.

```
CREATE TABLE logistica.Entrega
(
    IdEntrega BIGINT IDENTITY(1,1) NOT NULL,
    IdRuta BIGINT NOT NULL,
    FechaEntrega DATE NOT NULL,
    EstadoEntrega VARCHAR(50),
    TotalProductos INT,

    CONSTRAINT PK_ENTREGA PRIMARY KEY (IdEntrega),
    CONSTRAINT FK_RUTA FOREIGN KEY (IdRuta) REFERENCES logistica.Ruta(IdRuta),
    CONSTRAINT CHK_ESTADO CHECK (EstadoEntrega IN ('No entregado', 'En proceso', 'Entregado'))
)
```

Tabla logistica.DetalleEntrega:

Esta tabla se encarga de almacenar los detalles de las entregas realizadas, vinculando cada entrega con los productos almacenados. Los datos registrados incluyen: el identificador de la entrega, el identificador del almacenamiento del producto y la fecha del reporte de la entrega. La tabla establece relaciones con la tabla logistica.Entrega a través de la columna IdEntrega, y con la tabla almacen.AlmacenamientoProducto mediante la columna IdAlmacenamiento.

```
CREATE TABLE logistica.DetalleEntrega
(
    IdEntrega BIGINT NOT NULL,
    IdAlmacenamiento BIGINT NOT NULL,
    FechaReporte DATETIME NOT NULL,

    CONSTRAINT FK_ENTREGA FOREIGN KEY (IdEntrega) REFERENCES logistica.entrega(IdEntrega),
    CONSTRAINT FK_ALMACENAMIENTO FOREIGN KEY (IdAlmacenamiento) REFERENCES
    almacen.AlmacenamientoProducto(IdAlmacenamiento)
)
```

Automatización de datos mediante disparadores.

Cálculo automático del costo de entrega

El disparador TGR_COSTO_ENTREGA se activa después de insertar o actualizar un producto, calculando el costo con base en sus dimensiones y peso.

```
CREATE TRIGGER TGR_COSTO_ENTREGA
ON almacen.Producto
AFTER INSERT, UPDATE
AS
BEGIN

    DECLARE @CostoEnvio MONEY;
    DECLARE @LargoPaquete FLOAT;
    DECLARE @AnchoPaquete FLOAT;
    DECLARE @AltoPaquete FLOAT;
    DECLARE @PesoPaquete FLOAT;
    DECLARE @Dimensiones VARCHAR(100);
    DECLARE @IdProducto BIGINT;

    SELECT @Dimensiones = i.Dimensiones, @PesoPaquete = i.Peso, @IdProducto = i.IdProducto
    FROM inserted i;

    -- Se asume que la cadena de Dimensiones tiene el formato: Largo*Alto*Ancho
    SET @LargoPaquete = CONVERT(FLOAT, LEFT(@Dimensiones, CHARINDEX('*', @Dimensiones) - 1));
    SET @Dimensiones = RIGHT(@Dimensiones, LEN(@Dimensiones) - CHARINDEX('*', @Dimensiones));

    SET @AltoPaquete = CONVERT(FLOAT, LEFT(@Dimensiones, CHARINDEX('*', @Dimensiones) - 1));
    SET @Dimensiones = RIGHT(@Dimensiones, LEN(@Dimensiones) - CHARINDEX('*', @Dimensiones));

    SET @AnchoPaquete = CONVERT(FLOAT, @Dimensiones);
```

```

-- Se calcula el envío con la fórmula: (LargoPaquete+AltoPaquete+AnchoPaquete) * PesoPaquete
SET @CostoEnvio = ( @LargoPaquete + @AltoPaquete + @AnchoPaquete ) * @PesoPaquete;

--Se actualizan los campos correspondientes en la tabla
UPDATE almacen.Producto
SET CostoEntrega = @CostoEnvio
WHERE IdProducto = @IdProducto
END;

```

Gestión de capacidad de los anaqueles

El disparador TGR_ALMACENAMIENTO_PRODUCTO ajusta automáticamente la capacidad disponible de un anaquele cuando un producto es almacenado o retirado.

```

CREATE TRIGGER TGR_ALMACENAMIENTO_PRODUCTO
ON almacen.AlmacenamientoProducto
AFTER INSERT, Update
AS
BEGIN
    DECLARE @IdAnaquel BIGINT;

    SELECT @IdAnaquel = i.IdAnaquel
    FROM inserted i;

    UPDATE almacen.Anaquel
    SET Capacidad = Capacidad-1
    WHERE IdAnaquel = @IdAnaquel;
END;

```

Gestión de la Capacidad de los Anaqueles y Estado de la Entrega

El disparador TGR_DETALLE_ENTREGA se activa después de que se inserta un nuevo registro en la tabla logistica.DetalleEntrega. Su función es actualizar el estado de la entrega y ajustar la capacidad disponible de los anaqueles cuando un producto es entregado.

```

CREATE TRIGGER TGR_DETALLE_ENTREGA
ON logistica.DetalleEntrega
AFTER INSERT
AS
BEGIN
    DECLARE @IdEntrega BIGINT;
    DECLARE @IdAlmacenamiento BIGINT;

    SELECT @IdEntrega = i.IdEntrega, @IdAlmacenamiento = i.IdAlmacenamiento
    FROM inserted i;

    UPDATE logistica.Entrega
    SET EstadoEntrega = 'En proceso'
    WHERE IdEntrega = @IdEntrega

    UPDATE almacen.Anaquel
    SET Capacidad = Capacidad+1
    WHERE IdAnaquel = (SELECT IdAnaquel FROM almacen.AlmacenamientoProducto WHERE IdAlmacenamiento
    = @IdAlmacenamiento);
END;

```

Restricción de horarios de transportistas

Este disparador TGR_HORARIO_TRANSPORTISTA impide asignar rutas fuera del horario laboral de los transportistas.

```
CREATE TRIGGER TGR_HORARIO_TRANSPORTISTA
ON logistica.Ruta
INSTEAD OF INSERT
AS
BEGIN
    DECLARE @DentroHorario BIT;
    DECLARE @HoraSalidaRuta TIME;
    DECLARE @IdTransportista BIGINT;
    DECLARE @HoraEntradaTransp TIME;
    DECLARE @HoraSalidaTransp TIME;

    SELECT @HoraSalidaRuta = i.HoraSalida, @IdTransportista = i.IdTransportista
    FROM inserted i;

    SELECT @HoraEntradaTransp = t.HoraEntrada, @HoraSalidaTransp = t.HoraSalida
    FROM logistica.Transportista t
    WHERE t.IdTransportista = @IdTransportista;

    IF @HoraSalidaRuta BETWEEN @HoraEntradaTransp AND @HoraSalidaTransp
    BEGIN
        SET @DentroHorario = 1;
    END
    ELSE
    BEGIN
        SET @DentroHorario = 0;
    END

    -- Si no está dentro del horario del transportista
    IF @DentroHorario = 0
    BEGIN
        RAISERROR('El transportista no está disponible en el horario de la ruta asignada.', 16,
1);
        ROLLBACK TRANSACTION;
        RETURN;
    END

    -- Si está dentro del horario del transportista
    INSERT INTO logistica.Ruta (HoraSalida, HoraRegreso, IdTransportista, IdTransporte)
    SELECT i.HoraSalida, i.HoraRegreso, i.IdTransportista, i.IdTransporte
    FROM inserted i;
END;
```

Gestión de capacidad de transporte

Este disparador TGR_CAPACIDAD_TRANSPORTE asegura que los vehículos no excedan su capacidad máxima según el tipo.

```
CREATE TRIGGER TGR_CAPACIDAD_TRANSPORTE--no
ON logistica.Entrega
INSTEAD OF INSERT
AS
BEGIN
    DECLARE @IdTransporte BIGINT;
    DECLARE @CapacidadActual INT;
    DECLARE @TipoTransporte VARCHAR(50);
    DECLARE @CapacidadMaxima INT;
    DECLARE @TotalPaquetes INT;
    DECLARE @IdRuta BIGINT;

    SELECT @IdRuta = i.IdRuta
    FROM inserted i;

    SELECT @IdTransporte = r.IdTransporte
    FROM logistica.Ruta r
    WHERE r.IdRuta = @IdRuta;

    SELECT @TipoTransporte = t.Transporte, @CapacidadActual = t.Capacidad
    FROM logistica.Transporte t
    WHERE t.IdTransporte = @IdTransporte;

    IF @TipoTransporte = 'Camión'
        SET @CapacidadMaxima = 15;
    ELSE IF @TipoTransporte = 'Camioneta'
        SET @CapacidadMaxima = 10;
    ELSE IF @TipoTransporte = 'Coche'
        SET @CapacidadMaxima = 5;
    ELSE IF @TipoTransporte = 'Motocicleta'
        SET @CapacidadMaxima = 2;

    -- Se verifica si el transporte tiene la capacidad
    IF @CapacidadActual + 1 > @CapacidadMaxima
        BEGIN
            RAISERROR('El transporte no tiene la capacidad suficiente para llevar esta
entrega.', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END
    ELSE
        BEGIN
            -- Se actualiza la capacidad del transporte
            UPDATE logistica.Transporte
            SET Capacidad = Capacidad + 1
            WHERE IdTransporte = @IdTransporte;

            -- Se inserta la Entrega
            INSERT INTO logistica.Entrega (IdRuta, FechaEntrega, EstadoEntrega, TotalProductos)
            SELECT i.IdRuta, i.FechaEntrega, i.EstadoEntrega, i.TotalProductos
            FROM inserted i;
        END;
END;
```

Disparador para Incrementar el Total de Productos

Este disparador TGR_TOTAL_PRODUCTOS actualiza el campo TotalProductos en la tabla logistica.Entrega cada vez que se inserta un nuevo detalle de entrega en logistica.DetalleEntrega. Incrementa el total de productos asociados a una entrega.

```
CREATE TRIGGER TGR_TOTAL_PRODUCTOS
ON logistica.DetalleEntrega
AFTER INSERT
AS
BEGIN
    -- Declaración de variables
    DECLARE @IdEntrega BIGINT;

    -- Obtener el IdEntrega del registro insertado
    SELECT @IdEntrega = i.IdEntrega
    FROM inserted i;

    -- Incrementar el total de productos de la entrega correspondiente
    UPDATE logistica.Entrega
    SET TotalProductos = TotalProductos + 1
    WHERE IdEntrega = @IdEntrega;
END;
SELECT * FROM logistica.Entrega
```

Disparador para Decrementar el Total de Productos

Este disparador TGR_DECREMENTA_TOTAL_PRODUCTOS actualiza el campo TotalProductos en la tabla logistica.Entrega cada vez que se elimina un detalle de entrega de la tabla logistica.DetalleEntrega. Decrementa el total de productos asociados a la entrega.

```
CREATE TRIGGER TGR_DECREMENTA_TOTAL_PRODUCTOS
ON logistica.DetalleEntrega
AFTER DELETE
AS
BEGIN
    -- Evita mensajes adicionales
    SET NOCOUNT ON;

    -- Actualiza el TotalProductos de la tabla Entrega
    UPDATE logistica.Entrega
    SET TotalProductos = TotalProductos - 1
    WHERE IdEntrega IN (SELECT d.IdEntrega FROM deleted d);
END;
```

Estructura del sistema de software desarrollado en C#

Menú del Almacenamiento:

El programa "Menú del Almacenamiento" es una aplicación de escritorio desarrollada en C# utilizando Windows Forms, diseñada para gestionar diferentes módulos de un sistema de almacenamiento y logística. A través de una interfaz intuitiva, permite acceder a formularios específicos como *Proveedor*, *Cliente*, *Anaquele*, *Transporte*, y otros, cada uno de los cuales gestiona datos relacionados con dichas entidades. Cada botón del menú principal abre el formulario correspondiente, facilitando la visualización, inserción y administración de información de manera eficiente. Esto organiza el flujo de trabajo y centraliza el control de los datos en un solo sistema.

Inicio del código:

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace ProyectoAlmacenamiento
{
    public partial class Menu del almacenamiento : Form
    {
        public Menu del almacenamiento()
        {
            InitializeComponent();
        }

        private void button1_Click(object sender, EventArgs e)
        {
            Proveedor tablaProveedor = new Proveedor();
            tablaProveedor.Show();
        }

        private void button2_Click(object sender, EventArgs e)
        {
            Cliente tablaCliente = new Cliente();
            tablaCliente.Show();
        }

        private void button3_Click(object sender, EventArgs e)
        {
            Anaquele tablaAnaquele = new Anaquele();
            tablaAnaquele.Show();
        }

        private void button4_Click(object sender, EventArgs e)
        {
            Transporte tablaTransporte = new Transporte();
            tablaTransporte.Show();
        }

        private void button5_Click(object sender, EventArgs e)
        {
            Transportista tablaTransportista = new Transportista();
            tablaTransportista.Show();
        }
    }
}
```

```

private void button6_Click(object sender, EventArgs e)
{
    DomicilioCliente tablaDomicilioCliente = new DomicilioCliente();
    tablaDomicilioCliente.Show();
}

private void button7_Click(object sender, EventArgs e)
{
    Producto tablaProducto = new Producto();
    tablaProducto.Show();
}

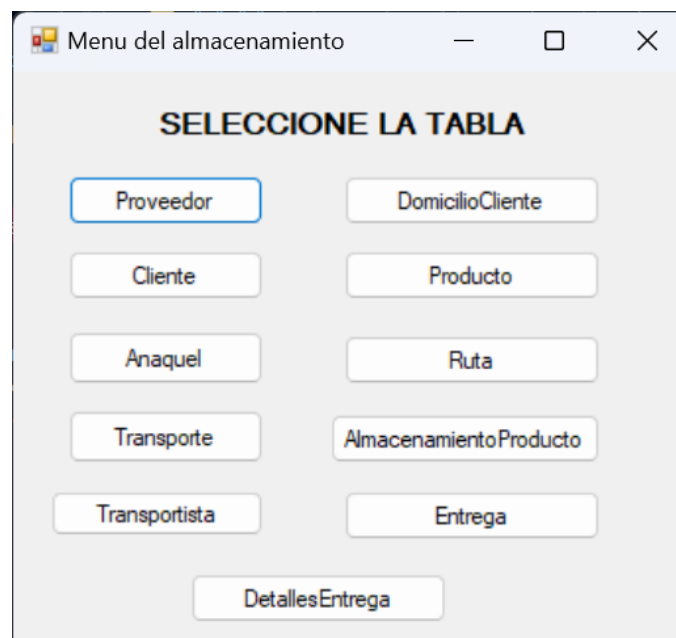
private void button8_Click(object sender, EventArgs e)
{
    Ruta tablaRuta = new Ruta();
    tablaRuta.Show();
}

private void button9_Click(object sender, EventArgs e)
{
    AlmacenamientoProducto tablaAlmacenamientoProducto = new AlmacenamientoProducto();
    tablaAlmacenamientoProducto.Show();
}
private void button10_Click(object sender, EventArgs e)
{
    Entrega tablaEntrega = new Entrega();
    tablaEntrega.Show();
}

private void button11_Click(object sender, EventArgs e)
{
    DetalleEntrega tablaEntregaDetalles = new DetalleEntrega();
    tablaEntregaDetalles.Show();
}
}
}

```

Visualización del formulario Menú del almacenamiento:



Conexión en la base de datos:

Es clase estática llamada Conexion, que centraliza la configuración y apertura de la conexión a una base de datos SQL Server.

Esta clase utiliza una cadena de conexión fija (connectionString) para conectarse a un servidor local llamado DESKTOP HEOM77M\SQLEXPRESS y a la base de datos llamada Almacenamiento. La autenticación se realiza mediante seguridad integrada de Windows (integrated security=true). El método estático GetConnection() crea una nueva instancia de SqlConnection con la cadena de conexión predefinida y la devuelve. Esta clase permite reutilizar la lógica de conexión a la base de datos en diferentes partes del programa.

```
public static class Conexion
{
    private const string connectionString = "Server = DESKTOP-HEOM77M\\SQLEXPRESS; " +
        "DATABASE = Almacenamiento; integrated security=true";

    public static SqlConnection GetConnection()
    {
        var connection = new SqlConnection(connectionString);
        return connection;
    }
}
```

Formulario Proveedor

Este código para gestionar registros de proveedores en una base de datos. La clase Proveedor es un formulario que permite realizar operaciones (Insertar, Consultar, Modificar y Eliminar) sobre la tabla almacen.Proveedor en la base de datos. A continuación, se explican las funcionalidades clave:

```
public partial class Proveedor : Form
{
    SqlConnection nuevaConexion = Conexion.GetConnection();

    string IdProveedor = "";
    string nombre_proveedor = "";
    string telefono_proveedor = "";
    string correo_proveedor = "";
    string domicilio_fiscal_proveedor = "";
    String telefono = "";
    String correo = "";
    public Proveedor()
    {
        InitializeComponent();
        ConsultaDatos();
    }
}
```

Consulta de Datos:

El método ConsultaDatos llena el control TablaProveedor (un DataGridView) con los registros existentes en la tabla almacen.Proveedor.

```
public void ConsultaDatos()
{
```

```

nuevaConexion.Open();
TablaProveedor.DataSource = null;
String selectInfo = "SELECT * FROM almacen.proveedor";
SqlCommand scm = new SqlCommand(selectInfo, nuevaConexion);
SqlDataReader lector = scm.ExecuteReader();

if (TablaProveedor.Columns.Count == 0)
{
    TablaProveedor.Columns.Add("IdProveedor", "ID");
    TablaProveedor.Columns.Add("Nombre_proveedor", "Nombre");
    TablaProveedor.Columns.Add("Telefono_proveedor", "Teléfono");
    TablaProveedor.Columns.Add("Correo_proveedor", "Correo");
    TablaProveedor.Columns.Add("Domicilio_fiscal_proveedor", "Domicilio Fiscal");
}

TablaProveedor.Rows.Clear();

int i = 0;
while (lector.Read())
{
    TablaProveedor.Rows.Add();
    TablaProveedor.Rows[i].Cells[0].Value = lector["IdProveedor"].ToString();
    TablaProveedor.Rows[i].Cells[1].Value = lector["NombreProveedor"].ToString();
    TablaProveedor.Rows[i].Cells[2].Value = lector["TelefonoProveedor"].ToString();
    TablaProveedor.Rows[i].Cells[3].Value = lector["CorreoProveedor"].ToString();
    TablaProveedor.Rows[i].Cells[4].Value = lector["DomicilioFiscal"].ToString();
    i++;
}

lector.Close();
nuevaConexion.Close();
}

```

Inserción de Registros:

El método InsertaDato inserta un nuevo registro en la tabla después de verificar que ni el correo ni el teléfono ya estén registrados (usando las funciones CorreoExiste y TelefonoExiste).

```

public void InsertaDato()
{
    nombre_proveedor = textNombre_Proveedor.Text;
    telefono_proveedor = textTelefono_Proveedor.Text;
    correo_proveedor = textCorreo_Proveedor.Text;
    domicilio_fiscal_proveedor = textDomicilio_Proveedor.Text;

    nuevaConexion.Open();

    // Verificar si el correo ya existe
    if (CorreoExiste(correo_proveedor, nuevaConexion))
    {
        MessageBox.Show("El correo ya está registrado.");
    }
    else if (TelefonoExiste(telefono_proveedor, nuevaConexion)) // Verificar si el
teléfono ya existe
    {
        MessageBox.Show("El teléfono ya está registrado.");
    }
    else
    {
        string insertInfo = "INSERT INTO almacen.Proveedor(NombreProveedor, " +
            "TelefonoProveedor, CorreoProveedor, DomicilioFiscal) VALUES ('" +
nombre_proveedor + "'," +

```

```

        "" + telefono_proveedor + "','" + correo_proveedor + "','" +
domicilio_fiscal_proveedor + "' )";

        SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion);
        cm.ExecuteNonQuery();
    }

    nombre_proveedor = "";
    telefono_proveedor = "";
    correo_proveedor = "";
    domicilio_fiscal_proveedor = "";

    nuevaConexion.Close();
}

// Función para verificar si el correo ya existe en la base de datos
public bool CorreoExiste(string correo, SqlConnection conexion)
{
    string query = "SELECT COUNT(1) FROM almacen.Proveedor WHERE CorreoProveedor =
@Correo";

    using (SqlCommand command = new SqlCommand(query, conexion))
    {
        command.Parameters.AddWithValue("@Correo", correo);

        try
        {
            int count = (int)command.ExecuteScalar();
            return count > 0; // Si el resultado es mayor a 0, significa que ya existe
        }
        catch (SqlException ex)
        {
            MessageBox.Show("Error al verificar el correo: " + ex.Message);
            return false;
        }
    }
}

// Función para verificar si el teléfono ya existe en la base de datos
public bool TelefonoExiste(string telefono, SqlConnection conexion)
{
    string query = "SELECT COUNT(1) FROM almacen.Proveedor WHERE TelefonoProveedor =
@Telefono";

    using (SqlCommand command = new SqlCommand(query, conexion))
    {
        command.Parameters.AddWithValue("@Telefono", telefono);

        try
        {
            int count = (int)command.ExecuteScalar();
            return count > 0; // Si el resultado es mayor a 0, significa que ya existe
        }
        catch (SqlException ex)
        {
            MessageBox.Show("Error al verificar el teléfono: " + ex.Message);
            return false;
        }
    }
}
}

```

Modificación de Registros:

El método ModificarDato actualiza los datos de un proveedor seleccionado. Realiza validaciones

para evitar duplicidad de correo o teléfono si estos valores son modificados.

```
private void ModificarDato(String telefono, String correo)
{
    nombre_proveedor = textNombre_Proveedor.Text;
    telefono_proveedor = textTelefono_Proveedor.Text;
    correo_proveedor = textCorreo_Proveedor.Text;
    domicilio_fiscal_proveedor = textDomicilio_Proveedor.Text;

    nuevaConexion.Open();

    // que cheque que se modificó

    bool modificoCorreo = correo.Equals(correo_proveedor);
    bool modificoTelefono = telefono.Equals(telefono_proveedor);

    if (CorreoExiste(correo_proveedor, nuevaConexion) && !modificoCorreo)
    {
        MessageBox.Show("El correo ya está registrado.");
    }
    else if (TelefonoExiste(telefono_proveedor, nuevaConexion) && !modificoTelefono) //
Verificar si el teléfono ya existe
    {
        MessageBox.Show("El teléfono ya está registrado.");
    }
    else
    {
        string insertInfo = "UPDATE almacen.Proveedor SET NombreProveedor = '" +
nombre_proveedor + "', TelefonoProveedor = '" +
        telefono_proveedor + "', CorreoProveedor = '" + correo_proveedor + "',
DomicilioFiscal = '" + domicilio_fiscal_proveedor + "' WHERE IdProveedor = '" + IdProveedor +
        "'";

        SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion);
        cm.ExecuteNonQuery();
    }

    nombre_proveedor = "";
    telefono_proveedor = "";
    correo_proveedor = "";
    domicilio_fiscal_proveedor = "";

    nuevaConexion.Close();
}
```

Eliminación de Registros:

El método RemoveDato elimina un registro de la base de datos identificado por IdProveedor.

```
private void RemoveDato()
{
    nuevaConexion.Open();

    string insertInfo = "DELETE FROM almacen.Proveedor WHERE IdProveedor = '" +
IdProveedor + "'";

    SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion);
    cm.ExecuteNonQuery();
}
```

```

        nuevaConexion.Close();

    }

    private void Proveedor_Load(object sender, EventArgs e)
    {

    }

    private void botonClickEliminar_Click(object sender, EventArgs e)
    {
        RemoverDato();
        ConsultaDatos();
        IdProveedor = "";
        textNombre_Proveedor.Clear();
        textTelefono_Proveedor.Clear();
        textCorreo_Proveedor.Clear();
        textDomicilio_Proveedor.Clear();
    }

    private void botonClickModificar_Click(object sender, EventArgs e)
    {

        ModificarDato(telefono, correo);
        ConsultaDatos();
        IdProveedor = "";
        textNombre_Proveedor.Clear();
        textTelefono_Proveedor.Clear();
        textCorreo_Proveedor.Clear();
        textDomicilio_Proveedor.Clear();
    }

    private void botonClickInsertar_Click(object sender, EventArgs e)
    {
        InsertaDato();
        ConsultaDatos();
        textNombre_Proveedor.Clear();
        textTelefono_Proveedor.Clear();
        textCorreo_Proveedor.Clear();
        textDomicilio_Proveedor.Clear();
    }

```

El evento `TablaProveedor_CellContentClick` permite capturar los datos de una fila seleccionada en el `DataGridView`.

```

private void TablaProveedor_CellContentClick_1(object sender, DataGridViewCellEventArgs e)
{
    if (e.RowIndex >= 0)
    {
        // Obtén la marca seleccionada
        DataGridViewRow filaSeleccionada = TablaProveedor.Rows[e.RowIndex];

        // Obtén el valor de una celda específica (por ejemplo, la primera transporte)
        IdProveedor = filaSeleccionada.Cells[0].Value.ToString();
        textNombre_Proveedor.Text = filaSeleccionada.Cells[1].Value.ToString();
        textTelefono_Proveedor.Text = filaSeleccionada.Cells[2].Value.ToString();
        textCorreo_Proveedor.Text = filaSeleccionada.Cells[3].Value.ToString();
        textDomicilio_Proveedor.Text = filaSeleccionada.Cells[4].Value.ToString();
        telefono = filaSeleccionada.Cells[2].Value.ToString();
        correo = filaSeleccionada.Cells[3].Value.ToString();
    }
}

```

```

    }
}

```

Formulario Cliente:

La clase **Cliente** extiende de **Form** y define funcionalidades como consulta, inserción, modificación y eliminación de registros en la tabla **logistica.Cliente**. Al iniciarse el formulario, se ejecuta el método **ConsultaDatos**, que carga los datos de los clientes desde la base de datos en un control **DataGridView** llamado **TablaCliente**. Los métodos **InsertaDato**, **ModificarDato** y **RemoveDato** permiten agregar nuevos registros, actualizar clientes existentes y eliminar registros, respectivamente, validando previamente que no existan conflictos como duplicados en correo o teléfono mediante las funciones **CorreoExiste** y **TelefonoExiste**. Los eventos de botones manejan las interacciones del usuario, como insertar, modificar o eliminar, y actualizan dinámicamente la tabla. Además, la selección de una fila en el **DataGridView** llena los campos de texto con la información del cliente correspondiente, facilitando la edición de datos. El uso de conexiones SQL se realiza cuidadosamente con apertura y cierre explícito para evitar bloqueos de recursos.

```

public partial class Cliente : Form
{
    SqlConnection nuevaConexion = Conexion.GetConnection();

    string IdCliente = "";
    string nombre_cliente = "";
    string telefono_cliente = "";
    string correo_cliente = "";
    String telefono = "";
    String correo = "";
    public Cliente()
    {
        InitializeComponent();
        ConsultaDatos();
    }

    public void ConsultaDatos()
    {
        nuevaConexion.Open();
        TablaCliente.DataSource = null;
        String selectInfo = "SELECT * FROM logistica.Cliente";
        SqlCommand scm = new SqlCommand(selectInfo, nuevaConexion);
        SqlDataReader lector = scm.ExecuteReader();

        if (TablaCliente.Columns.Count == 0)
        {
            TablaCliente.Columns.Add("IdCliente", "ID");
            TablaCliente.Columns.Add("Nombre_cliente", "Nombre");
            TablaCliente.Columns.Add("Telefono_cliente", "Teléfono");
            TablaCliente.Columns.Add("Correo_cliente", "Correo");
        }

        TablaCliente.Rows.Clear();

        int i = 0;
        while (lector.Read())
        {
            TablaCliente.Rows.Add();
            TablaCliente.Rows[i].Cells[0].Value = lector["IdCliente"].ToString();
            TablaCliente.Rows[i].Cells[1].Value = lector["NombreCliente"].ToString();

```

```

        TablaCliente.Rows[i].Cells[2].Value = lector["TelefonoCliente"].ToString();
        TablaCliente.Rows[i].Cells[3].Value = lector["CorreoCliente"].ToString();
        i++;
    }

    lector.Close();
    nuevaConexion.Close();
}

```

```

public void InsertaDato()
{
    nombre_cliente = textNombre_Cliente.Text;
    telefono_cliente = textTelefono_Cliente.Text;
    correo_cliente = textCorreo_Cliente.Text;

    nuevaConexion.Open();

    // Verificar si el correo ya existe
    if (CorreoExiste(correo_cliente, nuevaConexion))
    {
        MessageBox.Show("El correo ya está registrado.");
    }
    else if (TelefonoExiste(telefono_cliente, nuevaConexion)) // Verificar si el teléfono
ya existe
    {
        MessageBox.Show("El teléfono ya está registrado.");
    }
    else
    {
        string insertInfo = "INSERT INTO logistica.Cliente(NombreCliente, " +
            "TelefonoCliente, CorreoCliente ) VALUES ('" + nombre_cliente + "','" +
            "'" + telefono_cliente + "','" + correo_cliente + "'" );

        SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion);
        cm.ExecuteNonQuery();
    }

    nombre_cliente = "";
    telefono_cliente = "";
    correo_cliente = "";

    nuevaConexion.Close();
}

```

// Función para verificar si el correo ya existe en la base de datos

```

public bool CorreoExiste(string correo, SqlConnection conexion)
{
    string query = "SELECT COUNT(1) FROM logistica.Cliente WHERE CorreoCliente = @Correo";

    using (SqlCommand command = new SqlCommand(query, conexion))
    {
        command.Parameters.AddWithValue("@Correo", correo);

        try
        {
            int count = (int)command.ExecuteScalar();
            return count > 0; // Si el resultado es mayor a 0, significa que ya existe
        }
        catch (SqlException ex)
        {
            MessageBox.Show("Error al verificar el correo: " + ex.Message);
        }
    }
}

```

```

        return false;
    }
}

// Función para verificar si el teléfono ya existe en la base de datos
public bool TelefonoExiste(string telefono, SqlConnection conexion)
{
    string query = "SELECT COUNT(1) FROM logistica.Cliente WHERE TelefonoCliente = @Telefono";

    using (SqlCommand command = new SqlCommand(query, conexion))
    {
        command.Parameters.AddWithValue("@Telefono", telefono);

        try
        {
            int count = (int)command.ExecuteScalar();
            return count > 0; // Si el resultado es mayor a 0, significa que ya existe
        }
        catch (SqlException ex)
        {
            MessageBox.Show("Error al verificar el teléfono: " + ex.Message);
            return false;
        }
    }
}

private void ModificarDato(String telefono, String correo)
{
    nombre_cliente = textNombre_Cliente.Text;
    telefono_cliente = textTelefono_Cliente.Text;
    correo_cliente = textCorreo_Cliente.Text;

    nuevaConexion.Open();

    // que cheque que se modificó

    bool modificoCorreo = correo.Equals(correo_cliente);
    bool modificoTelefono = telefono.Equals(telefono_cliente);

    if (CorreoExiste(correo_cliente, nuevaConexion) && !modificoCorreo)
    {
        MessageBox.Show("El correo ya está registrado.");
    }
    else if (TelefonoExiste(telefono_cliente, nuevaConexion) && !modificoTelefono) //
    Verificar si el teléfono ya existe
    {
        MessageBox.Show("El teléfono ya está registrado.");
    }
    else
    {
        string insertInfo = "UPDATE logistica.Cliente SET NombreCliente = '" +
        nombre_cliente + "', TelefonoCliente = '" +
        telefono_cliente + "', CorreoCliente = '" + correo_cliente + "' WHERE IdCliente
        = '" + IdCliente + "'";

        SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion);
        cm.ExecuteNonQuery();
    }

    nombre_cliente = "";
}

```

```

        telefono_cliente = "";
        correo_cliente = "";

        nuevaConexion.Close();
    }

    private void RemoverDato()
    {
        nuevaConexion.Open();

        string insertInfo = "DELETE FROM logistica.Cliente WHERE IdCliente = '" + IdCliente +
        """;

        SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion);
        cm.ExecuteNonQuery();

        nuevaConexion.Close();

    }

    private void Proveedor_Load(object sender, EventArgs e)
    {

    }

    private void botonClickEliminar_Click(object sender, EventArgs e)
    {
        RemoverDato();
        ConsultaDatos();
        IdCliente = "";
        textNombre_Cliente.Clear();
        textTelefono_Cliente.Clear();
        textCorreo_Cliente.Clear();
    }

    private void botonClickModificar_Click(object sender, EventArgs e)
    {
        ModificarDato(telefono, correo);
        ConsultaDatos();
        IdCliente = "";
        textNombre_Cliente.Clear();
        textTelefono_Cliente.Clear();
        textCorreo_Cliente.Clear();
    }

    private void botonClickInsertar_Click(object sender, EventArgs e)
    {
        InsertaDato();
        ConsultaDatos();
        textNombre_Cliente.Clear();
        textTelefono_Cliente.Clear();
        textCorreo_Cliente.Clear();
    }

    private void TablaCliente_CellContentClick_1(object sender, DataGridViewCellEventArgs e)
    {
        if (e.RowIndex >= 0)
        {
            // Obtén la marca seleccionada
            DataGridViewRow filaSeleccionada = TablaCliente.Rows[e.RowIndex];

            // Obtén el valor de una celda específica (por ejemplo, la primera transporte)

```

```

        IdCliente = filaSeleccionada.Cells[0].Value.ToString();
        textNombre_Cliente.Text = filaSeleccionada.Cells[1].Value.ToString();
        textTelefono_Cliente.Text = filaSeleccionada.Cells[2].Value.ToString();
        textCorreo_Cliente.Text = filaSeleccionada.Cells[3].Value.ToString();
        telefono = filaSeleccionada.Cells[2].Value.ToString();
        correo = filaSeleccionada.Cells[3].Value.ToString();
    }
}
}

```

Formulario Anaquel:

La clase Anaquel tiene varios métodos que gestionan la interacción con la base de datos para realizar operaciones sobre los anaqueles. El método ConsultaDatos() se encarga de obtener los registros de los anaqueles desde la base de datos y mostrarlos en el DataGridView. Realiza una consulta SQL para obtener todos los datos y luego los inserta en las filas de la tabla para su visualización. El método InsertaDato() permite insertar un nuevo anaquel en la base de datos, asegurándose de que los valores de columna, fila y nivel sean numéricos. Antes de realizar la inserción, verifica que el lugar no esté ocupado mediante el método LugarExiste(). Este último método comprueba si ya existe un registro en la base de datos con la misma combinación de columna, fila y nivel, retornando true si el lugar está ocupado y false si está libre. El método ModificarDato() se usa para actualizar un registro existente, comparando los valores actuales con los anteriores para asegurarse de que no se esté modificando un campo que ya está ocupado. Si la validación es exitosa, realiza la actualización en la base de datos. El método RemoveDato() elimina el registro del anaquel cuya identificación coincide con el IdAnaquel actual. Finalmente, el método TablaAnaquel_CellContentClick() se activa cuando el usuario hace clic en una celda del DataGridView, permitiendo recuperar y mostrar los valores de la fila seleccionada en los campos de texto para su posterior edición o eliminación.

```

public partial class Anaquel : Form
{
    SqlConnection nuevaConexion = Conexion.GetConnection();

    string IdAnaquel = "";
    string columna = "";
    string fila = "";
    string nivel = "";
    string capacidad = "";
    String filaanterior = "";
    String nivelanterior = "";
    String columnaanterior = "";

    public Anaquel()
    {
        InitializeComponent();
        ConsultaDatos();
    }

    public void ConsultaDatos()
    {
        nuevaConexion.Open();
        TablaAnaquel.DataSource = null;
        String selectInfo = "SELECT * FROM almacen.Anaquel";
        SqlCommand scm = new SqlCommand(selectInfo, nuevaConexion);
    }
}

```

```

SqlDataReader lector = scm.ExecuteReader();

if (TablaAnaquel.Columns.Count == 0)
{
    TablaAnaquel.Columns.Add("IdAnaquel", "ID");
    TablaAnaquel.Columns.Add("Nivel", "Nivel");
    TablaAnaquel.Columns.Add("Fila", "Fila");
    TablaAnaquel.Columns.Add("Columna", "Columna");
    TablaAnaquel.Columns.Add("Capacidad", "Capacidad");
}

TablaAnaquel.Rows.Clear();

int i = 0;
while (lector.Read())
{
    TablaAnaquel.Rows.Add();
    TablaAnaquel.Rows[i].Cells[0].Value = lector["IdAnaquel"].ToString();
    TablaAnaquel.Rows[i].Cells[1].Value = lector["Nivel"].ToString();
    TablaAnaquel.Rows[i].Cells[2].Value = lector["Fila"].ToString();
    TablaAnaquel.Rows[i].Cells[3].Value = lector["Columna"].ToString();
    TablaAnaquel.Rows[i].Cells[4].Value = lector["Capacidad"].ToString();

    i++;
}

lector.Close();
nuevaConexion.Close();
}

public void InsertaDato()
{
    columna = textColumna.Text;
    fila = textFila.Text;
    nivel = textNivel.Text;
    capacidad = textCapacidad.Text;

    nuevaConexion.Open();

    if (LugarExiste(columna, fila, nivel, nuevaConexion))
    {
        MessageBox.Show("El lugar esta ocupado.");
    }
    else if (double.TryParse(fila, out _) && double.TryParse(nivel, out _) &&
double.TryParse(columna, out _))
    {
        string insertInfo = "INSERT INTO almacen.Anaquel(Columna, " +
        "Fila, Nivel, Capacidad) VALUES ('" + columna + "','" +
        "'" + fila + "','" + nivel + "','" + capacidad + "')";

        SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion);
        cm.ExecuteNonQuery();
    }
    else
    {
        MessageBox.Show("Los campos fila, columna y nivel deben ser numéricos.");
    }
}

```

```

        columna = "";
        fila = "";
        nivel = "";
        capacidad = "";

        nuevaConexion.Close();
    }

    public bool LugarExiste(string columna, string fila, string nivel, SqlConnection conexion)
    {
        string query = "SELECT COUNT(1) FROM almacen.Anaquel WHERE Columna = @columna AND Fila = @fila AND Nivel = @nivel";

        using (SqlCommand command = new SqlCommand(query, conexion))
        {
            command.Parameters.AddWithValue("@columna", columna);
            command.Parameters.AddWithValue("@fila", fila);
            command.Parameters.AddWithValue("@nivel", nivel);

            try
            {
                int count = (int)command.ExecuteScalar();
                return count > 0; // Si el resultado es mayor a 0, significa que ya existe
            }
            catch (SqlException ex)
            {
                MessageBox.Show("Error al verificar el campos del lugar: " + ex.Message);
                return false;
            }
        }
    }

    private void ModificarDato()
    {
        columna = textColumna.Text;
        fila = textFila.Text;
        nivel = textNivel.Text;
        capacidad = textCapacidad.Text;

        nuevaConexion.Open();

        // que cheque que se modificó

        bool modificoNivel = nivelanterior.Equals(nivel);
        bool modificoColumna = columnaanterior.Equals(columna);
        bool modificoFila = filaanterior.Equals(fila);

        if (LugarExiste(columna, fila, nivel, nuevaConexion) && !modificoNivel
            || LugarExiste(columna, fila, nivel, nuevaConexion) && !modificoColumna
            || LugarExiste(columna, fila, nivel, nuevaConexion) && !modificoFila)
        {
            MessageBox.Show("El lugar esta ocupado.");
        }
        else if (double.TryParse(fila, out _) && double.TryParse(nivel, out _) &&
double.TryParse(columna, out _))
        {
            string insertInfo = "UPDATE almacen.Anaquel SET Columna = '" + columna + "', Fila
= '" +
                fila + "', Nivel = '" + nivel + "', Capacidad = '" + capacidad + "' WHERE
IdAnaquel = '" + IdAnaquel + "'";

```

```

        SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion);
        cm.ExecuteNonQuery();
    }
    else
    {
        MessageBox.Show("Los campos fila, columna y nivel deben ser numéricos.");
    }

    columna = "";
    fila = "";
    nivel = "";
    capacidad = "";

    nuevaConexion.Close();
}

private void RemoverDato()
{
    nuevaConexion.Open();

    string insertInfo = "DELETE FROM almacen.Anaquel WHERE IdAnaquel = '" + IdAnaquel +
""";

    SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion);
    cm.ExecuteNonQuery();

    nuevaConexion.Close();
}

private void botonClickEliminar_Click(object sender, EventArgs e)
{
    RemoverDato();
    ConsultaDatos();
    IdAnaquel = "";
    textCapacidad.Clear();
    textColumna.Clear();
    textFila.Clear();
    textNivel.Clear();
}

private void botonClickModificar_Click(object sender, EventArgs e)
{
    ModificarDato();
    ConsultaDatos();
    IdAnaquel = "";
    textCapacidad.Clear();
    textColumna.Clear();
    textFila.Clear();
    textNivel.Clear();
}

private void botonClickInsertar_Click(object sender, EventArgs e)
{
    InsertaDato();
    ConsultaDatos();
    textCapacidad.Clear();
    textColumna.Clear();
    textFila.Clear();
    textNivel.Clear();
}

```

```

private void TablaAnaquel_CellContentClick(object sender, DataGridViewCellEventArgs e)
{
    if (e.RowIndex >= 0)
    {
        // Obtén la marca seleccionada
        DataGridViewRow filaSeleccionada = TablaAnaquel.Rows[e.RowIndex];

        // Obtén el valor de una celda específica (por ejemplo, la primera transporte)
        IdAnaquel = filaSeleccionada.Cells[0].Value.ToString();
        textColumna.Text = filaSeleccionada.Cells[3].Value.ToString();
        textFila.Text = filaSeleccionada.Cells[2].Value.ToString();
        textNivel.Text = filaSeleccionada.Cells[1].Value.ToString();
        textCapacidad.Text = filaSeleccionada.Cells[4].Value.ToString();
        columnaanterior = filaSeleccionada.Cells[3].Value.ToString();
        filaanterior = filaSeleccionada.Cells[2].Value.ToString();
        nivelanterior = filaSeleccionada.Cells[1].Value.ToString();
    }
}
}

```

Formulario Transporte:

La clase Transporte gestiona los registros de vehículos de transporte en una base de datos. El método ConsultaDatos() recupera los datos de transporte desde la base de datos y los muestra en una tabla. En el método EsCapacidadValida(), se verifica que la capacidad indicada para un tipo de transporte no exceda los límites permitidos. Cuando se agrega un nuevo registro con InsertaDato(), se valida que la capacidad sea adecuada y que la placa no esté duplicada antes de proceder con la inserción. El método PlacaExiste() se utiliza para comprobar si una placa ya está registrada en la base de datos. En el caso de modificaciones, ModificarDato() actualiza los registros existentes tras validar la capacidad y la placa. Para eliminar un registro, el método RemoveDato() elimina un transporte de la base de datos utilizando su identificador. Los métodos asociados a los botones de la interfaz (botonClickInsertar_Click, botonClickModificar_Click, botonClickEliminar_Click) gestionan la interacción con el usuario, realizando la acción correspondiente y actualizando la vista de datos. Finalmente, el método TablaTransporte_CellContentClick() permite que al seleccionar una fila en la tabla, los datos del transporte se muestren en los campos de entrada para su edición o eliminación.

```

public partial class Transporte : Form
{
    SqlConnection nuevaConexion = Conexion.GetConnection();

    string IdTransporte = "";
    string transporte = "";
    string marca = "";
    string modelo = "";
    string color = "";
    string placa = "";
    string capacidad = "";
    String placaAnterior = "";

    public Transporte()
    {
        InitializeComponent();
        ConsultaDatos();
        comboBox1.Items.Add("Camión");
    }
}

```

```

comboBox1.Items.Add("Camioneta");
comboBox1.Items.Add("Motocicleta");
comboBox1.Items.Add("Coche");

}

private bool EsCapacidadValida(string tipoTransporte, string capacidad)
{
    int capacidadMaxima = 0;
    bool esValida = false;

    // Validar que la capacidad sea un número
    if (!int.TryParse(capacidad, out int capacidadNum))
    {
        MessageBox.Show("Capacidad debe ser un número.");
        return false; // No es un número válido
    }

    // Verificar el tipo de transporte y el límite de capacidad
    if (tipoTransporte == "Camión")
    {
        capacidadMaxima = 15;
        esValida = capacidadNum <= capacidadMaxima;
    }
    else if (tipoTransporte == "Camioneta")
    {
        capacidadMaxima = 10;
        esValida = capacidadNum <= capacidadMaxima;
    }
    else if (tipoTransporte == "Coche")
    {
        capacidadMaxima = 5;
        esValida = capacidadNum <= capacidadMaxima;
    }
    else if (tipoTransporte == "Motocicleta")
    {
        capacidadMaxima = 2;
        esValida = capacidadNum <= capacidadMaxima;
    }

    if (!esValida)
    {
        MessageBox.Show($"Capacidad máxima para {tipoTransporte} es {capacidadMaxima}.");
    }

    return esValida;
}

public void ConsultaDatos()
{
    nuevaConexion.Open();
    TablaTransporte.DataSource = null;
    String selectInfo = "SELECT * FROM logistica.Transporte";
    SqlCommand scm = new SqlCommand(selectInfo, nuevaConexion);
    SqlDataReader lector = scm.ExecuteReader();

    if (TablaTransporte.Columns.Count == 0)
    {
        TablaTransporte.Columns.Add("IdTransporte", "ID");
        TablaTransporte.Columns.Add("Transporte", "Transporte");
        TablaTransporte.Columns.Add("Marca", "Marca");
        TablaTransporte.Columns.Add("Modelo", "Modelo");
        TablaTransporte.Columns.Add("Color", "Color");
    }
}

```

```

        TablaTransporte.Columns.Add("Placa", "Placa");
        TablaTransporte.Columns.Add("Capacidad", "Capacidad");

    }

    TablaTransporte.Rows.Clear();

    int i = 0;
    while (lector.Read())
    {
        TablaTransporte.Rows.Add();
        TablaTransporte.Rows[i].Cells[0].Value = lector["IdTransporte"].ToString();
        TablaTransporte.Rows[i].Cells[1].Value = lector["Transporte"].ToString();
        TablaTransporte.Rows[i].Cells[2].Value = lector["Marca"].ToString();
        TablaTransporte.Rows[i].Cells[3].Value = lector["Modelo"].ToString();
        TablaTransporte.Rows[i].Cells[4].Value = lector["Color"].ToString();
        TablaTransporte.Rows[i].Cells[5].Value = lector["Placa"].ToString();
        TablaTransporte.Rows[i].Cells[6].Value = lector["Capacidad"].ToString();

        i++;
    }

    lector.Close();
    nuevaConexion.Close();
}

public void InsertaDato()
{
    transporte = comboBox1.Text;
    marca = textMarca.Text;
    modelo = textModelo.Text;
    color = textColor.Text;
    placa = textPlaca.Text;
    capacidad = textCapacidad.Text;

    // Validar la capacidad antes de insertar
    if (!EsCapacidadValida(transporte, capacidad))
    {
        MessageBox.Show("Capacidad no válida para el tipo de transporte.");
        return; // Detener la inserción si la capacidad es incorrecta
    }

    nuevaConexion.Open();

    if (PlacaExiste(placa, nuevaConexion))
    {
        MessageBox.Show("Placa ya registrada.");
    }
    else if (transporte == "Camión" || transporte == "Camioneta" || transporte ==
"Motocicleta" || transporte == "Coche")
    {
        string insertInfo = "INSERT INTO logistica.Transporte(Transporte, Marca," +
"Modelo, Color, Placa, Capacidad) VALUES ('" + transporte + "','" +
        "'" + marca + "','" + modelo + "','" + color + "','" + placa + "','" + capacidad
+ "')";

        SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion);
        cm.ExecuteNonQuery();
    }
    else
    {
        MessageBox.Show("El transporte es incorrecto.");
    }
}

```

```

        transporte = "";
        marca = "";
        modelo = "";
        color = "";
        placa = "";
        capacidad = "";

        nuevaConexion.Close();
    }

    public bool PlacaExiste(string placa, SqlConnection conexion)
    {
        string query = "SELECT COUNT(1) FROM logistica.Transporte WHERE Placa = @Placa";

        using (SqlCommand command = new SqlCommand(query, conexion))
        {
            command.Parameters.AddWithValue("@Placa", placa);

            try
            {
                int count = (int)command.ExecuteScalar();
                return count > 0; // Si el resultado es mayor a 0, significa que ya existe
            }
            catch (SqlException ex)
            {
                MessageBox.Show("Error al verificar la placa: " + ex.Message);
                return false;
            }
        }
    }

    public void ModificarDato()
    {
        transporte = comboBox1.Text;
        marca = textMarca.Text;
        modelo = textModelo.Text;
        color = textColor.Text;
        placa = textPlaca.Text;
        capacidad = textCapacidad.Text;

        // Validar la capacidad antes de modificar
        if (!EsCapacidadValida(transporte, capacidad))
        {
            MessageBox.Show("Capacidad no válida para el tipo de transporte.");
            return; // Detener la modificación si la capacidad es incorrecta
        }

        nuevaConexion.Open();

        bool modificoPlaca = placaAnterior.Equals(placa);

        if (PlacaExiste(placa, nuevaConexion) && !modificoPlaca)
        {
            MessageBox.Show("Placa ya registrada.");
        }
        else if (transporte == "Camión" || transporte == "Camioneta" || transporte ==
"Motocicleta" || transporte == "Coche")
        {
            string updateInfo = "UPDATE logistica.Transporte SET Transporte = '" + transporte
+ "', Marca = '" +
                marca + "', Modelo = '" + modelo + "', Color = '" + color + "', Placa = '" + placa
+ "', Capacidad = '" + capacidad + "' WHERE IdTransporte = '" + IdTransporte + "'";

            SqlCommand cm = new SqlCommand(updateInfo, nuevaConexion);

```

```

        cm.ExecuteNonQuery();
    }
    else
    {
        MessageBox.Show("El transporte es incorrecto.");
    }

    transporte = "";
    marca = "";
    modelo = "";
    color = "";
    placa = "";
    capacidad = "";

    nuevaConexion.Close();
}

private void RemoverDato()
{
    nuevaConexion.Open();

    string insertInfo = "DELETE FROM logistica.Transporte WHERE IdTransporte = '" +
IdTransporte + "'";

    SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion);
    cm.ExecuteNonQuery();

    nuevaConexion.Close();

}

private void botonClickEliminar_Click(object sender, EventArgs e)
{
    RemoverDato();
    ConsultaDatos();
    IdTransporte = "";
    comboBox1.Text = "";
    textMarca.Clear();
    textModelo.Clear();
    textColor.Clear();
    textPlaca.Clear();
    textCapacidad.Clear();
}

private void botonClickModificar_Click(object sender, EventArgs e)
{
    ModificarDato();
    ConsultaDatos();
    IdTransporte = "";
    comboBox1.Text = "";
    textMarca.Clear();
    textModelo.Clear();
    textColor.Clear();
    textPlaca.Clear();
    textCapacidad.Clear();
}

private void botonClickInsertar_Click(object sender, EventArgs e)
{
    InsertaDato();
    ConsultaDatos();
}

```

```

        comboBox1.Text = "";
        textMarca.Clear();
        textModelo.Clear();
        textColor.Clear();
        textPlaca.Clear();
        textCapacidad.Clear();
    }

    private void TablaTransporte_CellContentClick(object sender, DataGridViewCellEventArgs e)
    {
        if (e.RowIndex >= 0)
        {
            // Obtén la marca seleccionada
            DataGridViewRow filaSeleccionada = TablaTransporte.Rows[e.RowIndex];

            // Obtén el valor de una celda específica (por ejemplo, la primera transporte)
            IdTransporte = filaSeleccionada.Cells[0].Value.ToString();
            comboBox1.Text = filaSeleccionada.Cells[1].Value.ToString();
            textMarca.Text = filaSeleccionada.Cells[2].Value.ToString();
            textModelo.Text = filaSeleccionada.Cells[3].Value.ToString();
            textColor.Text = filaSeleccionada.Cells[4].Value.ToString();
            textPlaca.Text = filaSeleccionada.Cells[5].Value.ToString();
            textCapacidad.Text = filaSeleccionada.Cells[6].Value.ToString();
            placaAnterior = filaSeleccionada.Cells[5].Value.ToString();

        }
    }
}

```

Formulario Transportista

La clase Transporte gestiona los registros de vehículos de transporte en una base de datos. El método ConsultaDatos() recupera los datos de transporte desde la base de datos y los muestra en una tabla. En el método EsCapacidadValida(), se verifica que la capacidad indicada para un tipo de transporte no exceda los límites permitidos. Cuando se agrega un nuevo registro con InsertaDato(), se valida que la capacidad sea adecuada y que la placa no esté duplicada antes de proceder con la inserción. El método PlacaExiste() se utiliza para comprobar si una placa ya está registrada en la base de datos. En el caso de modificaciones, ModificarDato() actualiza los registros existentes tras validar la capacidad y la placa. Para eliminar un registro, el método RemoveDato() elimina un transporte de la base de datos utilizando su identificador. Los métodos asociados a los botones de la interfaz (botonClickInsertar_Click, botonClickModificar_Click, botonClickEliminar_Click) gestionan la interacción con el usuario, realizando la acción correspondiente y actualizando la vista de datos. Finalmente, el método TablaTransporte_CellContentClick() permite que al seleccionar una fila en la tabla, los datos del transporte se muestren en los campos de entrada para su edición o eliminación.

```

public partial class Transportista : Form
{

    SqlConnection nuevaConexion = Conexion.GetConnection();

    string IdTransportista = "";
    string nombre_transportista = "";

```

```

string telefono_transportista = "";
string correo_transportista = "";
string horaentrada_transportista = "";
string horasalida_transportista = "";
String telefono = "";
String correo = "";
public Transportista()
{
    InitializeComponent();
    TimeSalida.Format = DateTimePickerFormat.Custom;
    TimeEntrada.Format = DateTimePickerFormat.Custom;
    TimeSalida.CustomFormat = "HH:mm";
    TimeEntrada.CustomFormat = "HH:mm";
    TimeSalida.ShowUpDown = true; // Esto oculta el calendario y solo permite seleccionar
la hora.
    TimeEntrada.ShowUpDown = true;
    TimeEntrada.Value = DateTime.Today.AddHours(00).AddMinutes(00);
    TimeSalida.Value = DateTime.Today.AddHours(00).AddMinutes(00);
    ConsultaDatos();
}

private void TimeSalida_ValueChanged(object sender, EventArgs e)
{
}

public void ConsultaDatos()
{
    nuevaConexion.Open();
    TablaTransportista.DataSource = null;
    String selectInfo = "SELECT * FROM logistica.Transportista";
    SqlCommand scm = new SqlCommand(selectInfo, nuevaConexion);
    SqlDataReader lector = scm.ExecuteReader();

    if (TablaTransportista.Columns.Count == 0)
    {
        TablaTransportista.Columns.Add("IdTransportista", "ID");
        TablaTransportista.Columns.Add("Nombre_transportista", "Nombre");
        TablaTransportista.Columns.Add("Telefono_transportista", "Teléfono");
        TablaTransportista.Columns.Add("Correo_transportista", "Correo");
        TablaTransportista.Columns.Add("HoraEntrada_transportista", "Hora de Entrada");
        TablaTransportista.Columns.Add("HoraSalida_transportista", "Hora de Salida");
    }
}

```

```

TablaTransportista.Rows.Clear();

int i = 0;
while (lector.Read())
{
    TablaTransportista.Rows.Add();
    TablaTransportista.Rows[i].Cells[0].Value = lector["IdTransportista"].ToString();
    TablaTransportista.Rows[i].Cells[1].Value =
lector["NombreTransportista"].ToString();
    TablaTransportista.Rows[i].Cells[2].Value =
lector["TelefonoTransportista"].ToString();
    TablaTransportista.Rows[i].Cells[3].Value =
lector["CorreoTransportista"].ToString();
    TablaTransportista.Rows[i].Cells[4].Value = lector["HoraEntrada"].ToString();
    TablaTransportista.Rows[i].Cells[5].Value = lector["HoraSalida"].ToString();

    i++;
}

lector.Close();
nuevaConexion.Close();
}

public void InsertaDato()
{
    nombre_transportista = textNombre_Transportista.Text;
    telefono_transportista = textTelefono_Transportista.Text;
    correo_transportista = textCorreo_Transportista.Text;
    DateTime selectedTime = TimeEntrada.Value;
    DateTime selectedTime2 = TimeSalida.Value;
    horaentrada_transportista = selectedTime.ToString("HH:mm");
    horasalida_transportista = selectedTime2.ToString("HH:mm");

    nuevaConexion.Open();

    // Verificar si el correo ya existe
    if (CorreoExiste(correo_transportista, nuevaConexion))
    {
        MessageBox.Show("El correo ya está registrado.");
    }
    else if (TelefonoExiste(telefono_transportista, nuevaConexion)) // Verificar si el
teléfono ya existe
    {
        MessageBox.Show("El teléfono ya está registrado.");
    }
    else

```

```

    {
        string insertInfo = "INSERT INTO logistica.Transportista(NombreTransportista, " +
            "TelefonoTransportista, CorreoTransportista, HoraEntrada, HoraSalida) VALUES ('" +
nombre_transportista + "'," +
            "'" + telefono_transportista + "', '" + correo_transportista + "' , '" +
horaentrada_transportista + "' , '" + horasalida_transportista + "' )";

        SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion);
        cm.ExecuteNonQuery();
    }

    nombre_transportista = "";
    telefono_transportista = "";
    correo_transportista = "";
    horaentrada_transportista = "";
    horasalida_transportista = "";

    nuevaConexion.Close();
}

public bool CorreoExiste(string correo, SqlConnection conexion)
{
    string query = "SELECT COUNT(1) FROM logistica.Transportista WHERE
CorreoTransportista = @Correo";

    using (SqlCommand command = new SqlCommand(query, conexion))
    {
        command.Parameters.AddWithValue("@Correo", correo);

        try
        {
            int count = (int)command.ExecuteScalar();
            return count > 0; // Si el resultado es mayor a 0, significa que ya existe
        }
        catch (SqlException ex)
        {
            MessageBox.Show("Error al verificar el correo: " + ex.Message);
            return false;
        }
    }
}

// Función para verificar si el teléfono ya existe en la base de datos
public bool TelefonoExiste(string telefono, SqlConnection conexion)
{
    string query = "SELECT COUNT(1) FROM logistica.Transportista WHERE
TelefonoTransportista = @Telefono";

```

```

using (SqlCommand command = new SqlCommand(query, conexion))
{
    command.Parameters.AddWithValue("@Telefono", telefono);

    try
    {
        int count = (int)command.ExecuteScalar();
        return count > 0; // Si el resultado es mayor a 0, significa que ya existe
    }
    catch (SqlException ex)
    {
        MessageBox.Show("Error al verificar el teléfono: " + ex.Message);
        return false;
    }
}
}

```

```

private void ModificarDato()
{
    nombre_transportista = textNombre_Transportista.Text;
    telefono_transportista = textTelefono_Transportista.Text;
    correo_transportista = textCorreo_Transportista.Text;
    DateTime selectedTime = TimeEntrada.Value;
    DateTime selectedTime2 = TimeSalida.Value;
    horaentrada_transportista = selectedTime.ToString("HH:mm");
    horasalida_transportista = selectedTime2.ToString("HH:mm");

    nuevaConexion.Open();

    // que cheque que se modificó

    bool modificoCorreo = correo.Equals(correo_transportista);
    bool modificoTelefono = telefono.Equals(telefono_transportista);

    if (CorreoExiste(correo_transportista, nuevaConexion) && !modificoCorreo)
    {
        MessageBox.Show("El correo ya está registrado.");
    }
    else if (TelefonoExiste(telefono_transportista, nuevaConexion) && !modificoTelefono)
    // Verificar si el teléfono ya existe
    {

```

```

        MessageBox.Show("El teléfono ya está registrado.");
    }
    else
    {

        string insertInfo = "UPDATE logistica.Transportista SET NombreTransportista = '" +
nombre_transportista + "', TelefonoTransportista = '" +
        telefono_transportista + "', CorreoTransportista = '" + correo_transportista + "',
HoraEntrada = '" + horaentrada_transportista + "', HoraSalida = '" + horasalida_transportista
        + "' WHERE IdTransportista = '" + IdTransportista + "'";

        SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion);
        cm.ExecuteNonQuery();
    }

    nombre_transportista = "";
    telefono_transportista = "";
    correo_transportista = "";
    horaentrada_transportista = "";
    horasalida_transportista = "";

    nuevaConexion.Close();

}

private void RemoverDato()
{

    nuevaConexion.Open();

    string insertInfo = "DELETE FROM logistica.Transportista WHERE IdTransportista = '" +
IdTransportista + "'";

    SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion);
    cm.ExecuteNonQuery();

    nuevaConexion.Close();

}

private void Proveedor_Load(object sender, EventArgs e)
{

}

```

```

private void botonClickEliminar_Click(object sender, EventArgs e)
{
    RemoverDato();
    ConsultaDatos();
    IdTransportista = "";
    textNombre_Transportista.Clear();
    textTelefono_Transportista.Clear();
    textCorreo_Transportista.Clear();
    TimeEntrada.Value = DateTime.Today.AddHours(00).AddMinutes(00);
    TimeSalida.Value = DateTime.Today.AddHours(00).AddMinutes(00);

}

```

```

private void botonClickModificar_Click(object sender, EventArgs e)
{
    ModificarDato();
    ConsultaDatos();
    IdTransportista = "";
    textNombre_Transportista.Clear();
    textTelefono_Transportista.Clear();
    textCorreo_Transportista.Clear();
    TimeEntrada.Value = DateTime.Today.AddHours(00).AddMinutes(00);
    TimeSalida.Value = DateTime.Today.AddHours(00).AddMinutes(00);

}

```

```

private void botonClickInsertar_Click(object sender, EventArgs e)
{
    InsertaDato();
    ConsultaDatos();
    textNombre_Transportista.Clear();
    textTelefono_Transportista.Clear();
    textCorreo_Transportista.Clear();
    TimeEntrada.Value = DateTime.Today.AddHours(00).AddMinutes(00);
    TimeSalida.Value = DateTime.Today.AddHours(00).AddMinutes(00);

}

```

```

e) private void TablaTransportista_CellContentClick(object sender, DataGridViewCellEventArgs
{
    if (e.RowIndex >= 0)
    {
        // Obtén la marca seleccionada
        DataGridViewRow filaSeleccionada = TablaTransportista.Rows[e.RowIndex];

```

```

        // Obtén el valor de una celda específica (por ejemplo, la primera transporte)
        IdTransportista = filaSeleccionada.Cells[0].Value.ToString();
        textNombre_Transportista.Text = filaSeleccionada.Cells[1].Value.ToString();
        textTelefono_Transportista.Text = filaSeleccionada.Cells[2].Value.ToString();
        textCorreo_Transportista.Text = filaSeleccionada.Cells[3].Value.ToString();
        TimeEntrada.Value =
Convert.ToDateTime(filaSeleccionada.Cells[4].Value.ToString());
        TimeSalida.Value = Convert.ToDateTime(filaSeleccionada.Cells[5].Value.ToString());
        telefono = filaSeleccionada.Cells[2].Value.ToString();
        correo = filaSeleccionada.Cells[3].Value.ToString();

    }
}

private void Transportista_Load(object sender, EventArgs e)
{

}
}

```

Formulario domicilio del cliente

La clase DomicilioCliente es un formulario de Windows que maneja la gestión de domicilios para clientes en una base de datos SQL. En el constructor, se inicializa la interfaz y se cargan los datos de clientes y domicilios, ajustando las columnas de la tabla para que se muestren correctamente. La función CargarClientes() consulta los clientes desde la base de datos y los muestra en un ComboBox para que el usuario pueda seleccionar uno. La función ConsultaDatos() carga la información de los domicilios asociados a los clientes en una tabla de datos. InsertaDato() permite agregar un nuevo domicilio para un cliente, verificando previamente que no exista un domicilio duplicado. La función NumeroInteriorExiste() verifica si ya existe un domicilio con los mismos datos. ModificarDato() permite actualizar la información de un domicilio si se selecciona un cliente y se modifican los campos, también validando duplicados. RemoverDato() elimina un domicilio seleccionado de la base de datos. Los botones botonClickEliminar_Click, botonClickModificar_Click, y botonClickInsertar_Click ejecutan las funciones de eliminación, modificación e inserción de datos, respectivamente, y luego actualizan la tabla. Finalmente, LimpiarCampos() limpia los campos del formulario y TablaDomCliente_CellContentClick() maneja la selección de filas en la tabla, llenando los campos con los datos correspondientes del domicilio seleccionado.

```

public partial class DomicilioCliente : Form
{
    SqlConnection nuevaConexion = Conexion.GetConnection();

    string IdCliente = "";
    string IdDomicilio = "";

```

```

string calle = "";
string colonia = "";
string codigoPostal = "";
string numeroExterior = "";
string numeroInterior = "";
String Ext = "";
String Int = "";
String Col = "";
String Cal = "";
String CP = "";

```

```

public DomicilioCliente()
{

```

```

    InitializeComponent();
    ConsultaDatos();
    CargarClientes();

```

```

    // Ajustar tamaño de columnas y celdas.
    TablaDomCliente.Columns[0].Width = 80;
    TablaDomCliente.Columns[1].Width = 160;
    TablaDomCliente.Columns[2].Width = 160;
    TablaDomCliente.Columns[3].Width = 70;
    TablaDomCliente.Columns[4].Width = 70;
    TablaDomCliente.Columns[5].Width = 80;

```

```

    TablaDomCliente.Columns[6].Width = 70;

```

```

    // Ajustar tamaño de columnas y celdas.

```

```

}

```

```

public void CargarClientes()
{

```

```

    try
    {

```

```

        nuevaConexion.Open();
        String selectInfo = "SELECT IdCliente, NombreCliente, TelefonoCliente FROM
logistica.Cliente ";
        SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion);
        SqlDataReader lector = cmd.ExecuteReader();

```

```

        comboBoxClientes.Items.Clear(); // Limpiar ComboBox
        while (lector.Read())

```

```

        {
            // Obtener el cliente y telefono
            string nombreCliente = lector["NombreCliente"].ToString();
            string telefonoCliente = lector["TelefonoCliente"].ToString();

            string clienteYdigitos = $"{nombreCliente}-{
telefonoCliente.Substring(telefonoCliente.Length - 3)}"; //sub string para obtener los ultimos
3 digitos

            comboBoxClientes.Items.Add(new KeyValuePair<string,
string>(lector["IdCliente"].ToString(), clienteYdigitos));
        }

        comboBoxClientes.DisplayMember = "Value"; // Mostrar el nombre en el ComboBox
        comboBoxClientes.ValueMember = "Key"; // El valor será el IdCliente

        lector.Close();
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error al cargar los clientes: " + ex.Message);
    }
    finally
    {
        nuevaConexion.Close();
    }
}

public void ConsultaDatos()
{
    nuevaConexion.Open();
    TablaDomCliente.DataSource = null;

    string selectInfo = "SELECT * FROM logistica.DomicilioCliente INNER JOIN
logistica.Cliente ON logistica.DomicilioCliente.IdCliente = logistica.Cliente.IdCliente ";

    SqlCommand scm = new SqlCommand(selectInfo, nuevaConexion);
    SqlDataReader lector = scm.ExecuteReader();

    // Limpiar las columnas previas
    if (TablaDomCliente.Columns.Count == 0)
    {
        TablaDomCliente.Columns.Add("IdDomicilio", "ID Domicilio");
    }
}

```

```

        TablaDomCliente.Columns.Add("IdCliente", "Cliente");
        TablaDomCliente.Columns.Add("Calle", "Calle");
        TablaDomCliente.Columns.Add("NumeroExt", "Número Exterior");
        TablaDomCliente.Columns.Add("NumeroInt", "Número Interior");
        TablaDomCliente.Columns.Add("Colonia", "Colonia");
        TablaDomCliente.Columns.Add("CodigoPostal", "CP");
    }

    TablaDomCliente.Rows.Clear();

    int i = 0;
    while (lector.Read())
    {
        TablaDomCliente.Rows.Add();
        TablaDomCliente.Rows[i].Cells[0].Value = lector["IdDomicilio"].ToString();
        string nombreCliente = lector["NombreCliente"].ToString();
        string telefonoCliente = lector["TelefonoCliente"].ToString();

        string clienteYdigitos = $"{nombreCliente}-{
{telefonoCliente.Substring(telefonoCliente.Length - 3)}}"; //sub string para obtener los ultimos
3 digitos

        TablaDomCliente.Rows[i].Cells[1].Value = clienteYdigitos;
        TablaDomCliente.Rows[i].Cells[2].Value = lector["Calle"].ToString();
        TablaDomCliente.Rows[i].Cells[3].Value = lector["NumeroExterior"].ToString();
        TablaDomCliente.Rows[i].Cells[4].Value = lector["NumeroInterior"].ToString();
        TablaDomCliente.Rows[i].Cells[5].Value = lector["Colonia"].ToString();
        TablaDomCliente.Rows[i].Cells[6].Value = lector["CodigoPostal"].ToString();
        i++;
    }

    lector.Close();

    nuevaConexion.Close();

}

public void InsertaDato()
{
    if (comboBoxClientes.SelectedItem != null)
    {
        var selectedClient = (KeyValuePair<string,
string>)comboBoxClientes.SelectedItem;
        IdCliente = selectedClient.Key; // Obtén el IdCliente del ComboBox
    }
}

```

```

        calle = textCalle_Cliente.Text;
        numeroExterior = textExt.Text; // Usar valores numéricos
        numeroInterior = textInt.Text; // Usar valores numéricos
        colonia = text_coloniaCliente.Text;
        codigoPostal = text_codigoPostal.Text;

        nuevaConexion.Open();

        if (NumeroInteriorExiste(IdCliente, numeroInterior, numeroExterior, calle,
            colonia, codigoPostal, nuevaConexion))
        {
            MessageBox.Show("El domicilio ya esta registrado.");
        }
        else
        {
            string insertInfo = "INSERT INTO logistica.DomicilioCliente(IdCliente,
                Calle, NumeroExterior, NumeroInterior, Colonia, CodigoPostal)" +
                " VALUES (@IdCliente, @Calle, @NumeroExt,
                @NumeroInt, @Colonia, @CodigoPostal)";

            SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion);
            cm.Parameters.AddWithValue("@IdCliente", IdCliente);
            cm.Parameters.AddWithValue("@Calle", calle);
            cm.Parameters.AddWithValue("@NumeroExt", numeroExterior);
            cm.Parameters.AddWithValue("@NumeroInt", numeroInterior);
            cm.Parameters.AddWithValue("@Colonia", colonia);
            cm.Parameters.AddWithValue("@CodigoPostal", codigoPostal);

            cm.ExecuteNonQuery();

            LimpiarCampos();
        }
    }
    else
    {
        MessageBox.Show("Por favor, selecciona un cliente.");
    }
    nuevaConexion.Close();
}

```

```

public bool NumeroInteriorExiste(string idcliente, string numint, string numext, string

```

```

calle, string colonia, string codigop, SqlConnection conexion)
{
    string query = "SELECT COUNT(1) FROM logistica.DomicilioCliente WHERE " +
        "IdCliente =@IdCliente AND NumeroInterior = @Numint AND NumeroExterior =
@Numext " +
        "AND Colonia = @colonia AND Calle = @calle ANDCodigoPostal = @codigop ";

    using (SqlCommand command = new SqlCommand(query, conexion))
    {
        command.Parameters.AddWithValue("@Idcliente", idcliente);
        command.Parameters.AddWithValue("@Numint", numint);
        command.Parameters.AddWithValue("@Numext", numext);
        command.Parameters.AddWithValue("@colonia", colonia);
        command.Parameters.AddWithValue("@calle", calle);
        command.Parameters.AddWithValue("@codigop", codigop);

        try
        {
            int count = (int)command.ExecuteScalar();
            return count > 0; // Si el resultado es mayor a 0, significa que ya existe
        }
        catch (SqlException ex)
        {
            MessageBox.Show("Error al verificar el numerointerior: " + ex.Message);
            return false;
        }
    }
}

private void ModificarDato()
{
    if (comboBoxClientes.SelectedItem != null)
    {
        var selectedClient = (KeyValuePair<string,
string>)comboBoxClientes.SelectedItem;
        IdCliente = selectedClient.Key; // Obtén el IdCliente del ComboBox

        calle = textCalle_Cliente.Text;
        numeroExterior = textExt.Text; // Usar valores numéricos
        numeroInterior = textInt.Text; // Usar valores numéricos
        colonia = text_coloniaCliente.Text;
        codigoPostal = text_codigoPostal.Text;

        nuevaConexion.Open();
    }
}

```

```

        bool modifikoInt = numeroInterior.Equals(Int);
        bool modifikoExt = numeroExterior.Equals(Ext);
        bool modifikoCol = colonia.Equals(Col);
        bool modifikoCal = calle.Equals(Cal);
        bool modifikoCP = codigoPostal.Equals(CP);

        bool isMod = false;

        if (!modifikoInt || !modifikoExt || !modifikoCol || !modifikoCal ||
!modifikoCP)
            isMod = true;

        if (NumeroInteriorExiste(IdCliente, numeroInterior, numeroExterior, calle,
colonia, codigoPostal, nuevaConexion) && isMod)
        {
            MessageBox.Show("El número interior ya está registrado.");
        }
        else
        {
            string updateInfo = "UPDATE logistica.DomicilioCliente SET IdCliente =
@IdCliente, Calle = @Calle, NumeroExterior = @NumeroExt, " +
                                "NumeroInterior = @NumeroInt, Colonia = @Colonia,
CodigoPostal = @CodigoPostal " +
                                "WHERE IdDomicilio = @IdDomicilio";

            SqlCommand cm = new SqlCommand(updateInfo, nuevaConexion);
            cm.Parameters.AddWithValue("@IdCliente", IdCliente);
            cm.Parameters.AddWithValue("@Calle", calle);
            cm.Parameters.AddWithValue("@NumeroExt", numeroExterior);
            cm.Parameters.AddWithValue("@NumeroInt", numeroInterior);
            cm.Parameters.AddWithValue("@Colonia", colonia);
            cm.Parameters.AddWithValue("@CodigoPostal", codigoPostal);
            cm.Parameters.AddWithValue("@IdDomicilio", IdDomicilio);

            cm.ExecuteNonQuery();

            LimpiarCampos();

```

```

        }
    }
    else
    {
        MessageBox.Show("Por favor, selecciona un cliente.");
    }
    nuevaConexion.Close();
}

private void RemoverDato()
{
    nuevaConexion.Open();

    string deleteInfo = "DELETE FROM logistica.DomicilioCliente WHERE IdDomicilio = @IdDomicilio";

    SqlCommand cm = new SqlCommand(deleteInfo, nuevaConexion);
    cm.Parameters.AddWithValue("@IdDomicilio", IdDomicilio);

    cm.ExecuteNonQuery();

    nuevaConexion.Close();
}

private void botonClickEliminar_Click(object sender, EventArgs e)
{
    RemoverDato();
    ConsultaDatos();
    LimpiarCampos();
}

private void botonClickModificar_Click(object sender, EventArgs e)
{
    ModificarDato();
    ConsultaDatos();
    LimpiarCampos();
}

private void botonClickInsertar_Click(object sender, EventArgs e)
{
    InsertaDato();
    ConsultaDatos();
}

private void LimpiarCampos()
{

```

```

        textCalle_Cliente.Clear();
        textExt.Clear();
        textInt.Clear();
        text_coloniaCliente.Clear();
        text_codigoPostal.Clear();
        comboBoxClientes.SelectedIndex = -1; // Deseleccionar el cliente
    }

```

e)

```

private void TablaDomCliente_CellContentClick(object sender, DataGridViewCellEventArgs
{
    if (e.RowIndex >= 0)
    {
        DataGridViewRow filaSeleccionada = TablaDomCliente.Rows[e.RowIndex];

        IdDomicilio = filaSeleccionada.Cells[0].Value.ToString();

        string idClienteSeleccionado = filaSeleccionada.Cells[1].Value.ToString();

        foreach (KeyValuePair<string, string> item in comboBoxClientes.Items)
        {
            if (item.Value == idClienteSeleccionado)
            {
                comboBoxClientes.SelectedItem = item;
                break;
            }
        }

        textCalle_Cliente.Text = filaSeleccionada.Cells[2].Value.ToString();
        textExt.Text = filaSeleccionada.Cells[3].Value.ToString();
        textInt.Text = filaSeleccionada.Cells[4].Value.ToString();
        text_coloniaCliente.Text = filaSeleccionada.Cells[5].Value.ToString();
        text_codigoPostal.Text = filaSeleccionada.Cells[6].Value.ToString();

        Cal = filaSeleccionada.Cells[2].Value.ToString();
        Ext = filaSeleccionada.Cells[3].Value.ToString();
        Int = filaSeleccionada.Cells[4].Value.ToString();
        Col = filaSeleccionada.Cells[5].Value.ToString();
        CP = filaSeleccionada.Cells[6].Value.ToString();
    }

}

private void DomicilioCliente_Load(object sender, EventArgs e)
{

```

```

    }
}
}

```

Formulario Producto:

La clase Producto incluyen ConsultaData(), que consulta y muestra la información de productos y sus relaciones en un DataGridView, InsertaDato() y ModificarDato(), que permiten agregar y actualizar productos respectivamente en la base de datos usando consultas SQL, y RemoveDato(), que elimina productos existentes. Además, incluye métodos como CargaProveedor() y CargaDomicilioCliente() para llenar los ComboBox con proveedores y domicilios, y TablaProducto_CellContentClick(), que carga datos de un producto seleccionado en el formulario para su edición. La clase también incluye un método LimpiarCampos() para reiniciar los campos del formulario. Esta clase facilita la gestión de productos mediante una interfaz gráfica, permitiendo consultar, insertar, modificar y eliminar productos en la base de datos de manera eficiente.

```

public partial class Producto : Form
{

    SqlConnection nuevaConexion = Conexion.GetConnection();
    string HoradeEntrada = "", ent = "", sal = "";
    string HoradeSalida = "";
    string IdProducto = "";
    public Producto()
    {
        InitializeComponent();
        ConsultaData();
        CargaProveedor();
        CargaDomicilioCliente();
        TablaProducto.Columns[0].Width = 40;
        TablaProducto.Columns[2].Width = 150;

    }
}

```

```

public void ConsultaData()
{
    try
    {

```

```

// Abre la conexión
nuevaConexion.Open();
TablaProducto.DataSource = null; // Limpia el DataGridView

// Define la consulta SQL
string selectInfo = "SELECT Proveedor.NombreProveedor,
Proveedor.TelefonoProveedor, DomicilioCliente.Calle, " +
"DomicilioCliente.NumeroInterior,
DomicilioCliente.NumeroExterior, Cliente.NombreCliente, Producto.NombreProducto, " +
"Producto.Peso, Producto.Dimensiones, Producto.CostoEntrega ,
Producto.IdProducto " +
"FROM almacen.Producto " +
"INNER JOIN almacen.Proveedor ON Producto.IdProveedor =
Proveedor.IdProveedor " +
"INNER JOIN logica.DomicilioCliente ON Producto.IdDomicilio
= DomicilioCliente.IdDomicilio " +
"INNER JOIN logica.Cliente ON DomicilioCliente.IdCliente =
Cliente.IdCliente";

SqlCommand scm = new SqlCommand(selectInfo, nuevaConexion);
SqlDataReader lector = scm.ExecuteReader();

// Configuración de columnas solo si es necesario
if (TablaProducto.Columns.Count == 0)
{
    TablaProducto.Columns.Add("IdProducto", "ID");
    TablaProducto.Columns.Add("Proveedor", "Proveedor");
    TablaProducto.Columns.Add("Domicilio", "Domicilio del Cliente");
    TablaProducto.Columns.Add("Producto", "Producto");
    TablaProducto.Columns.Add("Peso", "Peso (Kg)");
    TablaProducto.Columns.Add("Dimension", "Dimensiones (in)");
    TablaProducto.Columns.Add("Costo", "Costo de Entrega ($)");
}

TablaProducto.Rows.Clear(); // Limpia las filas del DataGridView

int i = 0;
while (lector.Read())
{
    // Obtiene y formatea los datos de cada columna
    string IdProducto = lector["IdProducto"].ToString();
    string nombreProveedor = lector["NombreProveedor"].ToString();
    string telProveedor = lector["TelefonoProveedor"].ToString();
    string Calle = lector["Calle"].ToString();

```

```

        string NumeroInterior = lector["NumeroInterior"].ToString();
        string NumeroExterior = lector["NumeroExterior"].ToString();
        string NombreCliente = lector["NombreCliente"].ToString();
        string NombreProducto = lector["NombreProducto"].ToString();
        string Dimension = lector["Dimensiones"].ToString();
        string Peso = lector["Peso"].ToString();

        string Costo = Convert.ToDecimal(lector["CostoEntrega"]).ToString("0.00");

        // Formatea las cadenas
        string ProveedorInfo = $"{nombreProveedor}-
{telProveedor.Substring(telProveedor.Length - 3)}";
        string DomicilioInfo = $"{Calle}-{NumeroInterior}-{NumeroExterior}
({NombreCliente})";

        // Agrega una nueva fila con los datos formateados
        TablaProducto.Rows.Add(IdProducto, ProveedorInfo,
DomicilioInfo, NombreProducto, Peso, Dimension, Costo);

        i++;
    }

    lector.Close(); // Cierra el lector
}
catch (Exception ex)
{
    MessageBox.Show("Error al consultar datos: " + ex.Message);
}
finally
{
    nuevaConexion.Close(); // Cierra la conexión
}
}

private void InsertaDato()
{
    string IdProveedor = ((KeyValuePair<string,
string>)comboBoxCliente.SelectedItem).Key;
    string IdDomicilio = ((KeyValuePair<string,
string>)comboBoxDomicilio.SelectedItem).Key;

```

```
nuevaConexion.Open();
```

```
        string insertInfo = "INSERT INTO almacen.Producto (IdProveedor,  
IdDomicilio, NombreProducto, Peso, Dimensiones) VALUES ('" + IdProveedor + "', '" + IdDomicilio  
+ "', '" + textNombreProducto.Text + "', '" + textPeso.Text +  
        "', '" + textDimensiones.Text + "')";
```

```
        SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion);  
        cm.ExecuteNonQuery();
```

```
nuevaConexion.Close();
```

```
}
```

```
public void CargaProveedor()  
{
```

```
    try  
    {
```

```
        nuevaConexion.Open();  
        String selectInfo = "SELECT IdProveedor, TelefonoProveedor, NombreProveedor  
FROM almacen.Proveedor ";  
        SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion);  
        SqlDataReader lector = cmd.ExecuteReader();
```

```
        comboBoxCliente.Items.Clear(); // Limpiar ComboBox
```

```
        while (lector.Read())  
        {
```

```
            // Obtener el cliente y telefono  
            string nombreProveedor = lector["NombreProveedor"].ToString();  
            string telProveedor = lector["TelefonoProveedor"].ToString();
```

```
            string proveedorInfo = $"{nombreProveedor}-  
{telProveedor.Substring(telProveedor.Length - 3)}";
```

```

        comboBoxCliente.Items.Add(new KeyValuePair<string,
string>(lector["IdProveedor"].ToString(), proveedorInfo));
    }

    comboBoxCliente.DisplayMember = "Value"; // Mostrar el nombre en el ComboBox
    comboBoxCliente.ValueMember = "Key"; // El valor será el IdCliente

    lector.Close();
}
catch (Exception ex)
{
    MessageBox.Show("Error al cargar los proveedores: " + ex.Message);
}
finally
{
    nuevaConexion.Close();
}

}

public void CargaDomicilioCliente()
{

    try
    {
        nuevaConexion.Open();
        String selectInfo = "SELECT IdDomicilio, Calle, NumeroInterior, NumeroExterior,
NombreCliente"+
        " FROM logistica.DomicilioCliente INNER JOIN logistica.Cliente ON
DomicilioCliente.IdCliente = Cliente.IdCliente";
        SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion);
        SqlDataReader lector = cmd.ExecuteReader();

        comboBoxDomicilio.Items.Clear(); // Limpiar ComboBox

        while (lector.Read())
        {

            string Calle = lector["Calle"].ToString();
            string NumeroInterior = lector["NumeroInterior"].ToString();
            string NumeroExterior = lector["NumeroExterior"].ToString();
            string NombreCliente = lector["NombreCliente"].ToString();

```

```

        string DomicilioInfo = $"{Calle}-{NumeroInterior}-{NumeroExterior}
({NombreCliente})";

        comboBoxDomicilio.Items.Add(new KeyValuePair<string,
string>(lector["IdDomicilio"].ToString(), DomicilioInfo));
    }

    comboBoxDomicilio.DisplayMember = "Value"; // Mostrar el nombre en el ComboBox
    comboBoxDomicilio.ValueMember = "Key"; // El valor será el IdCliente

    lector.Close();
}
catch (Exception ex)
{
    MessageBox.Show("Error al cargar la ruta: " + ex.Message);
}
finally
{
    nuevaConexion.Close();
}

}

private void ModificarDato()
{
    try
    {
        // Abrir conexión
        nuevaConexion.Open();

        // Obtener los datos actualizados de los controles
        string IdProveedor = ((KeyValuePair<string,
string>)comboBoxCliente.SelectedItem).Key;
        string IdDomicilio = ((KeyValuePair<string,
string>)comboBoxDomicilio.SelectedItem).Key;
        string nombreProducto = textNombreProducto.Text;
        string peso = textPeso.Text;
        string dimensiones = textDimensiones.Text;

        // Crear la consulta SQL de actualización
        string updateQuery = "UPDATE almacen.Producto SET " +
            "IdProveedor = @IdProveedor, " +
            "IdDomicilio = @IdDomicilio, " +
            "NombreProducto = @NombreProducto, " +
            "Peso = @Peso, " +
            "Dimensiones = @Dimensiones " +

```

```

        "WHERE IdProducto = @IdProducto";

        // Crear el comando SQL y asignar los parámetros
        SqlCommand cmd = new SqlCommand(updateQuery, nuevaConexion);
        cmd.Parameters.AddWithValue("@IdProveedor", IdProveedor);
        cmd.Parameters.AddWithValue("@IdDomicilio", IdDomicilio);
        cmd.Parameters.AddWithValue("@NombreProducto", nombreProducto);
        cmd.Parameters.AddWithValue("@Peso", peso);
        cmd.Parameters.AddWithValue("@Dimensiones", dimensiones);
        cmd.Parameters.AddWithValue("@IdProducto", IdProducto);

        // Ejecutar el comando
        int filasAfectadas = cmd.ExecuteNonQuery();

        if (filasAfectadas > 0)
        {

        }
        else
        {
            MessageBox.Show("No se encontró el producto para modificar.");
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error al modificar el producto: " + ex.Message);
    }
    finally
    {
        // Cerrar la conexión
        nuevaConexion.Close();
    }
}

private void RemoverDato()
{

    nuevaConexion.Open();

    string insertInfo = "DELETE FROM almacen.Producto WHERE IdProducto = '" +
IdProducto + "'";

    SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion);
    cm.ExecuteNonQuery();

    nuevaConexion.Close();
}

```

```
}
```

```
private void botonClickEliminar_Click(object sender, EventArgs e)
{
    RemoverDato();
    ConsultaDatos();

    LimpiarCampos();

}
```

```
private void botonClickModificar_Click(object sender, EventArgs e)
{

    ModificarDato();
    ConsultaDatos();

    LimpiarCampos();
}
```

```
private void botonClickInsertar_Click(object sender, EventArgs e)
{
    InsertaDato();
    ConsultaDatos();

    LimpiarCampos();
}
```

```
private void TablaProducto_CellContentClick(object sender, DataGridViewCellEventArgs e)
{
    if (e.RowIndex >= 0)
    {
        // Obtén la marca seleccionada
        DataGridViewRow filaSeleccionada = TablaProducto.Rows[e.RowIndex];

        // Obtén el valor de una celda específica (por ejemplo, la primera transporte)
        IdProducto = filaSeleccionada.Cells[0].Value.ToString();

        string idProveedorSeleccionado = filaSeleccionada.Cells[2].Value.ToString();

        foreach (KeyValuePair<string, string> item in comboBoxDomicilio.Items)
```

```

    {
        if (item.Value == idProveedorSeleccionado)
        {
            comboBoxDomicilio.SelectedItem = item;
            break;
        }
    }

    string idTDomicilioSeleccionado = filaSeleccionada.Cells[1].Value.ToString();

    foreach (KeyValuePair<string, string> item in comboBoxCliente.Items)
    {
        if (item.Value == idTDomicilioSeleccionado)
        {
            comboBoxCliente.SelectedItem = item;
            break;
        }
    }

    textNombreProducto.Text = filaSeleccionada.Cells[3].Value.ToString();
    textPeso.Text = filaSeleccionada.Cells[4].Value.ToString();
    textDimensiones.Text = filaSeleccionada.Cells[5].Value.ToString();

}

private void LimpiarCampos()
{
    comboBoxCliente.SelectedIndex = -1; ;
    comboBoxDomicilio.SelectedIndex = -1;
    textNombreProducto.Text = "";
    textPeso.Text = "";
    textDimensiones.Text = "";

}

}

```

Formulario Ruta

La clase Ruta **ConsultaDatos()** recupera los datos de las rutas almacenadas y los muestra en un DataGridView. **InsertaDato()** se encarga de agregar una nueva ruta, verificando previamente si ya existe mediante **RutaExiste()**, y si no es así, inserta los datos en la base de datos. **ModificarDato()** permite actualizar los datos de una ruta ya existente usando el id_ruta como referencia.

ComparaHoras() compara las horas de salida de dos rutas para verificar si se superponen, y **RestarHoras()** calcula la diferencia de tiempo entre dos horarios. **HorariosSeEmpalman()** verifica si los horarios de salida de las rutas coinciden o se solapan, evitando que las rutas se registren en el mismo intervalo de tiempo. En conjunto, estos métodos aseguran que las rutas se gestionen correctamente, evitando duplicados y conflictos de horario.

```
public partial class Ruta : Form
{

    SqlConnection nuevaConexion = Conexion.GetConnection();
    string HoradeEntrada = "", ent = "", sal = "";
    string HoradeSalida = "";
    string IdRuta = "";
    public Ruta()
    {
        InitializeComponent();

        TimeSalida.Format = DateTimePickerFormat.Custom;
        TimeEntrada.Format = DateTimePickerFormat.Custom;
        TimeSalida.CustomFormat = "HH:mm";
        TimeEntrada.CustomFormat = "HH:mm";
        TimeSalida.ShowUpDown = true; // Esto oculta el calendario y solo permite
seleccionar la hora.
        TimeEntrada.ShowUpDown = true;
        TimeEntrada.Value = DateTime.Today.AddHours(00).AddMinutes(00);
        TimeSalida.Value = DateTime.Today.AddHours(00).AddMinutes(00);
        ConsultaDatos();
        CargaTransporte();
        CargaTransportista();
        TablaRuta.Columns[0].Width = 40;

    }

    public static string RestarHoras(string hora1, string hora2)
    {
        // Convertir las cadenas a TimeSpan
        TimeSpan tiempo1 = TimeSpan.Parse(hora1);
```

```

        TimeSpan tiempo2 = TimeSpan.Parse(hora2);

        // Calcular la diferencia absoluta
        TimeSpan diferencia = tiempo1 > tiempo2 ? tiempo1 - tiempo2 : tiempo2 - tiempo1;

        // Convertir la diferencia a horas decimales
        return diferencia.TotalHours.ToString("F2");
    }

    public static bool ComparaHoras(string hora1, string hora2)
    {
        // Convertir las cadenas a TimeSpan
        TimeSpan tiempo1 = TimeSpan.Parse(hora1);
        TimeSpan tiempo2 = TimeSpan.Parse(hora2);

        // Calcular la diferencia absoluta
        bool diferencia = tiempo1 < tiempo2 ;

        // Convertir la diferencia a horas decimales
        return diferencia;
    }

    public void ConsultaDatos()
    {
        try
        {
            // Abre la conexión
            nuevaConexion.Open();
            TablaRuta.DataSource = null; // Limpia el DataGridView

            // Define la consulta SQL
            // Define la consulta SQL
            string selectInfo = "SELECT Ruta.IdRuta, Transportista.NombreTransportista,
Transportista.HoraEntrada, " +
                                "Transportista.HoraSalida AS HoraSalidaTransportista,
Transporte.Transporte, Transporte.Capacidad, Transporte.Modelo, " +
                                "Ruta.HoraSalida AS HoraSalidaRuta, Ruta.HoraRegreso " +
                                "FROM logistica.Ruta " +
                                "INNER JOIN logistica.Transporte ON Ruta.IdTransporte =
Transporte.IdTransporte " +
                                "INNER JOIN logistica.Transportista ON Ruta.IdTransportista
= Transportista.IdTransportista";

            SqlCommand scm = new SqlCommand(selectInfo, nuevaConexion);
            SqlDataReader lector = scm.ExecuteReader();

            // Configuración de columnas solo si es necesario
            if (TablaRuta.Columns.Count == 0)

```

```

{
    TablaRuta.Columns.Add("IdRuta", "ID");
    TablaRuta.Columns.Add("Transportista", "Transportista");
    TablaRuta.Columns.Add("Transporte", "Transporte");
    TablaRuta.Columns.Add("HoraSalida", "Hora de Salida");
    TablaRuta.Columns.Add("HoraRegreso", "Hora de Regreso");
}

TablaRuta.Rows.Clear(); // Limpia las filas del DataGridView

int i = 0;
while (lector.Read())
{
    // Obtiene y formatea los datos de cada columna
    string idRuta = lector["IdRuta"].ToString();
    string nombreTransportista = lector["NombreTransportista"].ToString();
    string horaEnt = lector["HoraEntrada"].ToString();
    string horaSalTransportista = lector["HoraSalidaTransportista"].ToString();
    string nombreTransporte = lector["Transporte"].ToString();
    string capacidad = lector["Capacidad"].ToString();
    string modelo = lector["Modelo"].ToString();
    string horaSalidaRuta = lector["HoraSalidaRuta"].ToString();
    string horaRegresoRuta = lector["HoraRegreso"].ToString();

    // Formatea las cadenas
    string transportistaInfo = $"{nombreTransportista}({RestarHoras(horaEnt,
horaSalTransportista)} horas)";
    string transporteInfo = $"{nombreTransporte}-{modelo}({capacidad})";

    // Agrega una nueva fila con los datos formateados
    TablaRuta.Rows.Add(idRuta, transportistaInfo, transporteInfo,
horaSalidaRuta, horaRegresoRuta);

    i++;
}

lector.Close(); // Cierra el lector
}
catch (Exception ex)
{
    MessageBox.Show("Error al consultar datos: " + ex.Message);
}
finally
{
    nuevaConexion.Close(); // Cierra la conexión
}
}

```

```

        public bool HorariosSeEmpalman(string idTransportista, string nuevaHoraEntrada, string
nuevaHoraSalida)
        {
            string query = @"
SELECT HoraSalida, HoraRegreso
FROM logistica.Ruta
WHERE IdTransportista = @IdTransportista
AND (@NuevaHoraEntrada >= HoraSalida AND @NuevaHoraEntrada < HoraRegreso)
OR (@NuevaHoraSalida > HoraSalida AND @NuevaHoraSalida <= HoraRegreso)
OR (HoraSalida >= @NuevaHoraEntrada AND HoraRegreso <= @NuevaHoraSalida));";

            using (SqlCommand command = new SqlCommand(query, nuevaConexion))
            {
                command.Parameters.AddWithValue("@IdTransportista", idTransportista);
                command.Parameters.AddWithValue("@NuevaHoraEntrada", nuevaHoraEntrada);
                command.Parameters.AddWithValue("@NuevaHoraSalida", nuevaHoraSalida);

                try
                {
                    SqlDataReader reader = command.ExecuteReader();
                    bool seEmpalma = reader.HasRows;
                    reader.Close();
                    return seEmpalma;
                }
                catch (SqlException ex)
                {
                    MessageBox.Show("Error al verificar los horarios: " + ex.Message);
                    return false;
                }
            }
        }
    }
}

```

```

private void InsertaDato()
{
    DateTime selectedTime = TimeEntrada.Value;
    DateTime selectedTime2 = TimeSalida.Value;
    HoradeEntrada = selectedTime.ToString("HH:mm");
    HoradeSalida = selectedTime2.ToString("HH:mm");

    string IdTransporte = ((KeyValuePair<string,
string>)comboBoxTransporte.SelectedItem).Key;
    string IdTransportista = ((KeyValuePair<string,
string>)comboBoxTransportista.SelectedItem).Key;
}

```

```

nuevaConexion.Open();

try
{
    if (ComparaHoras(HoradeSalida, HoradeEntrada))
    {
        if (RutaExiste(IdRuta, IdTransporte, IdTransportista, HoradeEntrada,
HoradeSalida, nuevaConexion))
        {
            MessageBox.Show("La ruta ya está registrada.");
        }
        else
        {
            if (!HorariosSeEmpalman(IdTransportista, HoradeSalida, HoradeEntrada))
            {
                string insertInfo = "INSERT INTO logistica.Ruta (IdTransporte,
IdTransportista, HoraSalida, HoraRegreso) " +
                "VALUES ('" + IdTransporte + "', '" +
IdTransportista + "', '" + HoradeSalida + "', '" + HoradeEntrada + "')";

                SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion);
                cm.ExecuteNonQuery();
            }
            else
            {
                MessageBox.Show("La hora se sobrepone.");
            }
        }
    }
    else
    {
        MessageBox.Show("La hora de salida no debe ser menor que la hora de
regreso.");
    }
}
catch (SqlException ex)
{
    MessageBox.Show("Problema con el query: " + ex.Message);
}
finally
{
    nuevaConexion.Close();
}
}

```

```

public void CargaTransporte()
{

    try
    {
        nuevaConexion.Open();
        String selectInfo = "SELECT IdTransporte, Transporte, Capacidad, Modelo FROM
logistica.Transporte ";
        SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion);
        SqlDataReader lector = cmd.ExecuteReader();

        comboBoxTransporte.Items.Clear(); // Limpiar ComboBox

        while (lector.Read())
        {
            // Obtener el cliente y telefono
            string nombreTransporte= lector["Transporte"].ToString();
            string modelo = lector["Modelo"].ToString();
            string capacidad = lector["Capacidad"].ToString();

            string cadena= $"{nombreTransporte}-{modelo}({capacidad})";

            comboBoxTransporte.Items.Add(new KeyValuePair<string,
string>(lector["IdTransporte"].ToString(), cadena));
        }

        comboBoxTransporte.DisplayMember = "Value"; // Mostrar el nombre en el ComboBox
        comboBoxTransporte.ValueMember = "Key"; // El valor será el IdCliente

        lector.Close();
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error al cargar la ruta: " + ex.Message);
    }
    finally
    {
        nuevaConexion.Close();
    }
}

public void CargaTransportista()
{

```

```

try
{
    nuevaConexion.Open();
    String selectInfo = "SELECT IdTransportista, NombreTransportista, HoraEntrada,
HoraSalida FROM logistica.Transportista ";
    SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion);
    SqlDataReader lector = cmd.ExecuteReader();

    comboBoxTransportista.Items.Clear(); // Limpiar ComboBox

    while (lector.Read())
    {

        string nombreTransportista = lector["NombreTransportista"].ToString();
        string HoraEnt = lector["HoraEntrada"].ToString();
        string HoraSal = lector["HoraSalida"].ToString();

        string trasportistayhoras = $"{nombreTransportista}({RestarHoras(HoraEnt,
HoraSal)} horas)";

        comboBoxTransportista.Items.Add(new KeyValuePair<string,
string>(lector["IdTransportista"].ToString(), trasportistayhoras));
    }

    comboBoxTransportista.DisplayMember = "Value"; // Mostrar el nombre en el
ComboBox
    comboBoxTransportista.ValueMember = "Key"; // El valor será el IdCliente

    lector.Close();
}
catch (Exception ex)
{
    MessageBox.Show("Error al cargar la ruta: " + ex.Message);
}
finally
{
    nuevaConexion.Close();
}
}

```

```

public bool RutaExiste(string idRuta, string IdTransporte, string IdTransportista,
string HoraEntrada, string HoraSalida, SqlConnection conexion)

```

```

{
    string query = "SELECT COUNT(1) FROM logistica.Ruta WHERE " +
        "IdRuta =@IdRuta AND HoraRegreso = @HoraEntrada AND HoraSalida= @HoraSalida " +
        "AND IdTransporte = @IdTransporte AND IdTransportista = @IdTransportista ";

    using (SqlCommand command = new SqlCommand(query, conexion))
    {
        command.Parameters.AddWithValue("@IdRuta", IdRuta);
        command.Parameters.AddWithValue("@HoraEntrada", HoraEntrada);
        command.Parameters.AddWithValue("@HoraSalida", HoraSalida);
        command.Parameters.AddWithValue("@IdTransportista", IdTransportista);
        command.Parameters.AddWithValue("@IdTransporte", IdTransporte);

        try
        {
            int count = (int)command.ExecuteScalar();
            return count > 0; // Si el resultado es mayor a 0, significa que ya existe
        }
        catch (SqlException ex)
        {
            MessageBox.Show("Error al verificar la ruta: " + ex.Message);
            return false;
        }
    }
}

private void ModificarDato()
{
    DateTime selectedTime = TimeEntrada.Value;
    DateTime selectedTime2 = TimeSalida.Value;
    HoradeEntrada = selectedTime.ToString("HH:mm");
    HoradeSalida = selectedTime2.ToString("HH:mm");

    if (comboBoxTransporte.SelectedItem != null && comboBoxTransportista.SelectedItem !=
null)
    {
        var selectedTrasporte = (KeyValuePair<string,
string>)comboBoxTransporte.SelectedItem;
        var IdTransporte = selectedTrasporte.Key;

        var selectedTransportista = (KeyValuePair<string,
string>)comboBoxTransportista.SelectedItem;
        var IdTransportista = selectedTransportista.Key;
    }
}

```

```

nuevaConexion.Open();

if (ComparaHoras(HoradeSalida, HoradeEntrada))
{

    bool modificoEnt = HoradeEntrada.Equals(ent);
    bool modificoSal = HoradeSalida.Equals(sal);

    bool isMod = false;

    if (!modificoEnt || !modificoSal)
        isMod = true;

    if (RutaExiste(IdRuta, IdTransporte, IdTransportista, HoradeEntrada,
HoradeSalida, nuevaConexion) && !isMod)
    {
        MessageBox.Show("La ruta ya está registrada.");
    }
    else
    {
        if (!HorariosSeEmpalman(IdTransportista, HoradeSalida, HoradeEntrada))
        {

            string updateInfo = "UPDATE logistica.Ruta SET IdTransporte =
@IdTransporte, IdTransportista = @IdTransportista, HoraSalida = @HoraSalida, " +
                                "HoraRegreso = @HoraRegreso WHERE IdRuta =
@IdRuta";

            SqlCommand cm = new SqlCommand(updateInfo, nuevaConexion);
            cm.Parameters.AddWithValue("@IdTransporte", IdTransporte);
            cm.Parameters.AddWithValue("@IdTransportista", IdTransportista);
            cm.Parameters.AddWithValue("@HoraSalida", HoradeSalida);
            cm.Parameters.AddWithValue("@HoraRegreso", HoradeEntrada);
            cm.Parameters.AddWithValue("@IdRuta", IdRuta);

            cm.ExecuteNonQuery();

```

```

        LimpiarCampos();

        }
        else
        {
            MessageBox.Show("La hora se sobrepone.");
        }

    }

    }else
    {
        MessageBox.Show("La hora de salida no debe ser menor que la hora de
regreso.");
    }
    nuevaConexion.Close();

}
else
{
    MessageBox.Show("Por favor, selecciona una ruta.");
}
}

private void RemoverDato()
{

    nuevaConexion.Open();

    string insertInfo = "DELETE FROM logistica.Ruta WHERE IdRuta = '" + IdRuta + "'";

    SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion);
    cm.ExecuteNonQuery();

    nuevaConexion.Close();

}

private void botonClickEliminar_Click(object sender, EventArgs e)
{

```

```

        RemoverDato();
        ConsultaDatos();

        LimpiarCampos();

    }

    private void botonClickModificar_Click(object sender, EventArgs e)
    {

        ModificarDato();
        ConsultaDatos();

        LimpiarCampos();
    }

    private void botonClickInsertar_Click(object sender, EventArgs e)
    {
        InsertaDato();
        ConsultaDatos();

        LimpiarCampos();
    }

    private void TablaRuta_CellContentClick(object sender, DataGridViewCellEventArgs e)
    {
        if (e.RowIndex >= 0)
        {
            // Obtén la marca seleccionada
            DataGridViewRow filaSeleccionada = TablaRuta.Rows[e.RowIndex];

            // Obtén el valor de una celda específica (por ejemplo, la primera transporte)
            IdRuta = filaSeleccionada.Cells[0].Value.ToString();

            string idTransportistaSeleccionado = filaSeleccionada.Cells[1].Value.ToString();

            foreach (KeyValuePair<string, string> item in comboBoxTransportista.Items)
            {
                if (item.Value == idTransportistaSeleccionado)
                {
                    comboBoxTransportista.SelectedItem = item;
                    break;
                }
            }
        }
    }

```

```

        string idTrasporteSeleccionado = filaSeleccionada.Cells[2].Value.ToString();

        foreach (KeyValuePair<string, string> item in comboBoxTransporte.Items)
        {
            if (item.Value == idTrasporteSeleccionado)
            {
                comboBoxTransporte.SelectedItem = item;
                break;
            }
        }

        TimeEntrada.Value =
Convert.ToDateTime(filaSeleccionada.Cells[4].Value.ToString());
        TimeSalida.Value =
Convert.ToDateTime(filaSeleccionada.Cells[3].Value.ToString());
        ent = TimeEntrada.Text = filaSeleccionada.Cells[4].Value.ToString();
        sal = TimeSalida.Text = filaSeleccionada.Cells[3].Value.ToString();

    }
}

private void LimpiarCampos()
{
    TimeEntrada.Value = DateTime.Today.AddHours(00).AddMinutes(00);
    TimeSalida.Value = DateTime.Today.AddHours(00).AddMinutes(00);
    comboBoxTransporte.SelectedIndex = -1; ;
    comboBoxTransportista.SelectedIndex = -1;

}

private void TimeSalida_ValueChanged(object sender, EventArgs e)
{
}

private void Ruta_Load(object sender, EventArgs e)
{
}
}

```

Formulario Almacenamiento Producto

El código contiene varios métodos clave para gestionar datos de productos almacenados en un sistema. **CargaAnaquel()** y **CargaProducto()** cargan los datos de los anaqueles y productos desde la base de datos para llenarlos en ComboBox, permitiendo al usuario seleccionar opciones. **ConsultaDatos()** consulta y muestra en un DataGridView los datos de almacenamiento, que incluyen información sobre los productos, fechas y ubicaciones. **InsertaDato()** permite agregar un nuevo registro de almacenamiento, validando posibles conflictos de fechas mediante **ValidaConflictoFechas()**, que verifica si el producto está almacenado en el mismo anaquel en el rango de fechas seleccionado. **ModificarDato()** actualiza un registro existente si las fechas son válidas y no hay conflictos. **RemoverDato()** elimina un registro de almacenamiento de la base de datos. **ComparaFechas()** valida que la fecha de salida no sea anterior a la de entrega. Finalmente, **TablaAlmacen_CellContentClick()** permite seleccionar un registro en la tabla y cargar su información en los campos correspondientes para su modificación o eliminación. Estos métodos forman un conjunto robusto para gestionar el almacenamiento de productos de manera eficiente y controlada.

```
public partial class AlmacenamientoProducto : Form
{
    SqlConnection nuevaConexion = Conexion.GetConnection();
    string IdAlmacenamiento = "", ent = "", sal = "";
    public AlmacenamientoProducto()
    {
        InitializeComponent();

        CargaAnaquel();
        CargaProducto();
        ConsultaDatos();
        TablaAlmacen.Columns[0].Width = 40;
        TablaAlmacen.Columns[2].Width = 150;
        FechaEntrega.MinDate = DateTime.Today;
        FechaSalida.MinDate = DateTime.Today;
    }

    public void CargaAnaquel()
    {
        try
        {
            nuevaConexion.Open();
            String selectInfo = "SELECT IdAnaquel, Nivel, Fila, Columna FROM
almacen.Anaquel";
            SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion);
            SqlDataReader lector = cmd.ExecuteReader();

            comboBoxAnaquel.Items.Clear(); // Limpiar ComboBox
```

```

while (lector.Read())
{
    // Obtener el cliente y telefono
    string Nivel = lector["Nivel"].ToString();
    string Fila = lector["Fila"].ToString();
    string Columna = lector["Columna"].ToString();

    string cadena = $"{Nivel} - {Fila} - {Columna}";

    comboBoxAnaquel.Items.Add(new KeyValuePair<string,
string>(lector["IdAnaquel"].ToString(), cadena));
}

comboBoxAnaquel.DisplayMember = "Value"; // Mostrar el nombre en el ComboBox
comboBoxAnaquel.ValueMember = "Key"; // El valor será el IdCliente

lector.Close();
}
catch (Exception ex)
{
    MessageBox.Show("Error al cargar referencias de anaquel: " + ex.Message);
}
finally
{
    nuevaConexion.Close();
}

}

public void CargaProducto()
{
    try
    {
        nuevaConexion.Open();
        String selectInfo = "SELECT IdProducto, Dimensiones, NombreProducto FROM
almacen.Producto";
        SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion);
        SqlDataReader lector = cmd.ExecuteReader();

        comboBoxProducto.Items.Clear(); // Limpiar ComboBox

        while (lector.Read())
        {
            // Obtener el cliente y telefono

```

```

        string Id = lector["IdProducto"].ToString();
        string Dimensiones = lector["Dimensiones"].ToString();
        string NombreProducto = lector["NombreProducto"].ToString();

        string cadena = $"{Id} - {NombreProducto} ({Dimensiones})";

        comboBoxProducto.Items.Add(new KeyValuePair<string,
string>(lector["IdProducto"].ToString(), cadena));
    }

    comboBoxProducto.DisplayMember = "Value"; // Mostrar el nombre en el ComboBox
    comboBoxProducto.ValueMember = "Key"; // El valor será el IdCliente

    lector.Close();
}
catch (Exception ex)
{
    MessageBox.Show("Error al cargar referencias de anaquel: " + ex.Message);
}
finally
{
    nuevaConexion.Close();
}

}

public void ConsultaDatos()
{
    try
    {
        // Abre la conexión
        nuevaConexion.Open();
        TablaAlmacen.DataSource = null; // Limpia el DataGridView

        // Define la consulta SQL
        // Define la consulta SQL
        string selectInfo = "SELECT IdAlmacenamiento, FechaEntrega, FechaSalida,
Dimensiones, AlmacenamientoProducto.IdProducto, " +
            "NombreProducto, Nivel, Fila, Columna FROM almacen.AlmacenamientoProducto "
+
            "INNER JOIN almacen.Producto ON
AlmacenamientoProducto.IdProducto = Producto.IdProducto " +
            "INNER JOIN almacen.Anaquel ON
AlmacenamientoProducto.IdAnaquel = Anaquel.IdAnaquel";

        SqlCommand scm = new SqlCommand(selectInfo, nuevaConexion);

```

```

SqlDataReader lector = scm.ExecuteReader();

// Configuración de columnas solo si es necesario
if (TablaAlmacen.Columns.Count == 0)
{
    TablaAlmacen.Columns.Add("IdAlmacenamiento", "ID");
    TablaAlmacen.Columns.Add("Anaquel", "Anaquel");
    TablaAlmacen.Columns.Add("Producto", "Producto");
    TablaAlmacen.Columns.Add("FechaSalida", "FechaSalida");
    TablaAlmacen.Columns.Add("FechaEntrega", "FechaEntrega");
}

TablaAlmacen.Rows.Clear(); // Limpia las filas del DataGridView

int i = 0;
while (lector.Read())
{
    // Obtiene y formatea los datos de cada columna
    string IdAlmacenamiento = lector["IdAlmacenamiento"].ToString();
    string FechaEntrega =
Convert.ToDateTime(lector["FechaEntrega"]).ToString("yyyy-MM-dd");
    string FechaSalida =
Convert.ToDateTime(lector["FechaSalida"]).ToString("yyyy-MM-dd");
    string Dimensiones = lector["Dimensiones"].ToString();
    string IdProducto = lector["IdProducto"].ToString();
    string NombreProducto = lector["NombreProducto"].ToString();
    string Nivel = lector["Nivel"].ToString();
    string Fila = lector["Fila"].ToString();
    string Columna = lector["Columna"].ToString();

    // Formatea las cadenas
    string anaquel = $"{Nivel} - {Fila} - {Columna}";
    string producto = $"{IdProducto} - {NombreProducto} ({Dimensiones})";

    // Agrega una nueva fila con los datos formateados
    TablaAlmacen.Rows.Add(IdAlmacenamiento, anaquel, producto, FechaSalida,
FechaEntrega);

    i++;
}

lector.Close(); // Cierra el lector
}
catch (Exception ex)
{
    MessageBox.Show("Error al consultar datos: " + ex.Message);
}

```

```

    }
    finally
    {
        nuevaConexion.Close(); // Cierra la conexión
    }
}

```

```

private void InsertaDato()
{

```

```

    string IdAnaquel = ((KeyValuePair<string,
string>)comboBoxAnaquel.SelectedItem).Key;
    string IdProducto = ((KeyValuePair<string,
string>)comboBoxProducto.SelectedItem).Key;

```

```

    if (ValidaConflictoFechas(IdProducto, FechaEntrega.Value, FechaSalida.Value))
    {
        MessageBox.Show("El producto ya está almacenado en este anaquel durante las
fechas seleccionadas.");
        return;
    }

```

```

    nuevaConexion.Open();

```

```

    if (ComparaFechas(FechaSalida.Value, FechaEntrega.Value))
    {

```

```

        string insertInfo = "INSERT INTO almacen.AlmacenamientoProducto(IdAnaquel,
IdProducto, FechaEntrega, FechaSalida) " +
            "VALUES (@IdAnaquel, @IdProducto, @FechaEntrega, @FechaSalida)";
        SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion);
        cm.Parameters.AddWithValue("@IdAnaquel", IdAnaquel);
        cm.Parameters.AddWithValue("@IdProducto", IdProducto);
        cm.Parameters.AddWithValue("@FechaEntrega", FechaEntrega.Value);
        cm.Parameters.AddWithValue("@FechaSalida", FechaSalida.Value);

        cm.ExecuteNonQuery();
    }
}

```

```

    }
    else
    {
        MessageBox.Show("La fecha de salida no debe ser mayor que la fecha de
regreso.");
    }

    nuevaConexion.Close();

}

```

```

private void ModificarDato()
{

    if (comboBoxAnaquel.SelectedItem != null && comboBoxProducto.SelectedItem != null)
    {
        var selectedAnaquel = (KeyValuePair<string,
string>)comboBoxAnaquel.SelectedItem;
        var IdAnaquel = selectedAnaquel.Key;

        var selectedProductos = (KeyValuePair<string,
string>)comboBoxProducto.SelectedItem;
        var IdProducto = selectedProductos.Key;

        if (ValidaConflictoFechas(IdProducto, FechaEntrega.Value, FechaSalida.Value))
        {
            MessageBox.Show("El producto ya está almacenado en este anaquel durante las
fechas seleccionadas.");
            return;
        }

        nuevaConexion.Open();

        if (ComparaFechas(FechaSalida.Value, FechaEntrega.Value))
        {

            bool modificoEnt = FechaEntrega.Equals(ent);
            bool modificoSal = FechaSalida.Equals(sal);

```

```

bool isMod = false;

if (!modificoEnt || !modificoSal)
    isMod = true;

if (!isMod)
{
    MessageBox.Show("El almacenamiento ya está registrado.");
}
else
{

    string updateInfo = "UPDATE almacen.AlmacenamientoProducto " +
        "SET IdAnaquel = @IdAnaquel, IdProducto = @IdProducto, " +
        "FechaEntrega = @FechaEntrega, FechaSalida = @FechaSalida " +
        "WHERE IdAlmacenamiento = @IdAlmacenamiento";

    SqlCommand cm = new SqlCommand(updateInfo, nuevaConexion);
    cm.Parameters.AddWithValue("@IdAnaquel", IdAnaquel);
    cm.Parameters.AddWithValue("@IdProducto", IdProducto);
    cm.Parameters.AddWithValue("@FechaEntrega", FechaEntrega.Value);
    cm.Parameters.AddWithValue("@FechaSalida", FechaSalida.Value);
    cm.Parameters.AddWithValue("@IdAlmacenamiento", IdAlmacenamiento);

    cm.ExecuteNonQuery();

    LimpiarCampos();

}

}
else
{

```

```

        MessageBox.Show("La hora de salida no debe ser menor que la hora de
regreso.");
    }
    nuevaConexion.Close();

}
else
{
    MessageBox.Show("Por favor, selecciona una ruta.");
}
}

private void RemoverDato()
{
    nuevaConexion.Open();

    string insertInfo = "DELETE FROM almacen.AlmacenamientoProducto WHERE
IdAlmacenamiento = '" + IdAlmacenamiento + "'";

    SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion);
    cm.ExecuteNonQuery();

    nuevaConexion.Close();

}

private void botonClickEliminar_Click(object sender, EventArgs e)
{
    RemoverDato();
    ConsultaDatos();

    LimpiarCampos();

}

private void botonClickModificar_Click(object sender, EventArgs e)
{
    ModificarDato();
    ConsultaDatos();
}

```

```

        LimpiarCampos();
    }

    private void botonClickInsertar_Click(object sender, EventArgs e)
    {

        InsertaDato();
        ConsultaDatos();

        LimpiarCampos();
    }

    private void TablaAlmacen_CellContentClick(object sender, DataGridViewCellEventArgs e)
    {
        if (e.RowIndex >= 0)
        {
            // Obtén la marca seleccionada
            DataGridViewRow filaSeleccionada = TablaAlmacen.Rows[e.RowIndex];

            // Obtén el valor de una celda específica (por ejemplo, la primera transporte)
            IdAlmacenamiento = filaSeleccionada.Cells[0].Value.ToString();

            string idAnaquelSeleccionado = filaSeleccionada.Cells[1].Value.ToString();

            foreach (KeyValuePair<string, string> item in comboBoxAnaquel.Items)
            {
                if (item.Value == idAnaquelSeleccionado)
                {
                    comboBoxAnaquel.SelectedItem = item;
                    break;
                }
            }

            string idProductoSeleccionado = filaSeleccionada.Cells[2].Value.ToString();

            foreach (KeyValuePair<string, string> item in comboBoxProducto.Items)
            {
                if (item.Value == idProductoSeleccionado)
                {
                    comboBoxProducto.SelectedItem = item;
                    break;
                }
            }
        }
    }

```

```

    }
    try
    {
        FechaEntrega.Value =
Convert.ToDateTime(filaSeleccionada.Cells[4].Value.ToString());
        FechaSalida.Value =
Convert.ToDateTime(filaSeleccionada.Cells[3].Value.ToString());
        ent = FechaEntrega.Text = filaSeleccionada.Cells[4].Value.ToString();
        sal = FechaSalida.Text = filaSeleccionada.Cells[3].Value.ToString();
    }
    catch
    {
        MessageBox.Show("No se puede cargar las Fechas anteriores");
    }
}
}

```

```

private bool ValidaConflictoFechas(string idProducto, DateTime fechaEntrega, DateTime
fechaSalida)
{
    bool conflicto = false;

    try
    {
        nuevaConexion.Open();

        string query = @"
        SELECT COUNT(*)
FROM almacen.AlmacenamientoProducto
WHERE IdProducto = @IdProducto
AND (
    @FechaSalida >= FechaSalida
    AND @FechaEntrega <= FechaEntrega
) OR @FechaSalida = FechaEntrega OR @FechaEntrega = FechaSalida;";

        SqlCommand cmd = new SqlCommand(query, nuevaConexion);
        cmd.Parameters.AddWithValue("@IdProducto", idProducto);
        cmd.Parameters.AddWithValue("@FechaEntrega", fechaEntrega);
        cmd.Parameters.AddWithValue("@FechaSalida", fechaSalida);

        int count = (int)cmd.ExecuteScalar();
    }
}

```

```

        conflicto = count > 0;
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error al validar conflicto de fechas: " + ex.Message);
    }
    finally
    {
        nuevaConexion.Close();
    }

    return conflicto;
}

private bool ComparaFechas(DateTime fechaSalida, DateTime fechaEntrega)
{
    if (fechaEntrega >= fechaSalida)
    {
        return true; // La fecha de entrada es válida
    }
    else
    {
        return false; // La fecha de entrada es inválida
    }
}

private void LimpiarCampos()
{
    FechaEntrega.Value = DateTime.Now;
    FechaSalida.Value = DateTime.Now;
    comboBoxAnaquel.SelectedIndex = -1; ;
    comboBoxProducto.SelectedIndex = -1;

}
}

```

Formulario Entrega

La clase Entrega contiene métodos que permiten consultar, insertar, modificar y eliminar registros en una tabla de entregas, además de gestionar el estado de la entrega según la fecha.

El formulario inicializa ciertos controles, como un DataGridView para mostrar las entregas, un ComboBox para seleccionar rutas y estados, y un DateTimePicker para seleccionar la fecha y hora de la entrega. Al cargar el formulario, se llaman a métodos que consultan datos de la base de datos y llenan los ComboBox y DataGridView.

El método ConsultaDatos realiza una consulta SQL para obtener las entregas y sus detalles (como

el ID de la entrega, nombre del transportista, ruta, estado y fecha de entrega), que luego se muestran en el DataGridView. También se actualizan los estados de las entregas automáticamente mediante el método ActualizarEstadoAutomático, que ajusta el estado de las entregas según la fecha actual.

Para insertar una nueva entrega, el método InsertaDato verifica si la combinación de ruta y fecha de entrega ya existe en la base de datos. Si no existe, inserta una nueva entrega con estado "En proceso" y un total de productos igual a 0.

El método ModificarDato permite actualizar una entrega existente. Verifica si los campos de ruta, fecha de entrega y estado han cambiado y, si es así, realiza una actualización en la base de datos. Si se realiza una modificación, también se limpia el formulario.

El método RemoverDato elimina una entrega seleccionada en el DataGridView. Los botones de la interfaz de usuario están vinculados a estos métodos de consulta, inserción, modificación y eliminación.

En resumen, el formulario permite gestionar entregas en un sistema logístico, incluyendo la actualización automática del estado de las entregas según la fecha, la inserción y modificación de entregas con validaciones, y la eliminación de registros seleccionados por el usuario.

```
public partial class Entrega : Form
{
    SqlConnection nuevaConexion = Conexion.GetConnection();
    string HoradeEntrada = "", ent = "";
    string HoradeSalida = "";
    string IdEntrega = "", Ruta = "", edo = "";

    public Entrega()
    {
        InitializeComponent();

        ConsultaDatos();
        CargaEstado();
        CargaRuta();
        TimeSalida.MinDate = DateTime.Today;
        TablaEntrega.Columns[0].Width = 40;
        TablaEntrega.Columns[1].Width = 110;

        TablaEntrega.Columns[2].Width = 110;
    }

    public void ConsultaDatos()
    {
        try
```

```

{

    ActualizarEstadoAutomático();
    // Abre la conexión
    nuevaConexion.Open();
    TablaEntrega.DataSource = null; // Limpia el DataGridView

    // Define la consulta SQL
    // Define la consulta SQL
    String selectInfo = "SELECT IdEntrega, EstadoEntrega, TotalProductos,
FechaEntrega, NombreTransportista, HoraRegreso, Ruta.HoraSalida " +
        "FROM logistica.Entrega " +
        "INNER JOIN logistica.Ruta ON Entrega.IdRuta = Ruta.IdRuta " +
        "INNER JOIN logistica.Transportista " +
        "ON Ruta.idTransportista = Transportista.idTransportista "
    ;

    SqlCommand scm = new SqlCommand(selectInfo, nuevaConexion);
    SqlDataReader lector = scm.ExecuteReader();

    // Configuración de columnas solo si es necesario
    if (TablaEntrega.Columns.Count == 0)
    {
        TablaEntrega.Columns.Add("IdEntrega", "ID");
        TablaEntrega.Columns.Add("Ruta", "Ruta");
        TablaEntrega.Columns.Add("FechaEntrega", "Fecha de Entrega");
        TablaEntrega.Columns.Add("Estado", "Estado");
        TablaEntrega.Columns.Add("Totalproductos", "Total de productos");
    }

    TablaEntrega.Rows.Clear(); // Limpia las filas del DataGridView

    int i = 0;
    while (lector.Read())
    {
        // Obtiene y formatea los datos de cada columna
        string idEntrega = lector["IdEntrega"].ToString();

        string fechaEntrega =
Convert.ToDateTime(lector["FechaEntrega"]).ToString("yyyy-MM-dd");
        string estado = lector["EstadoEntrega"].ToString();
        string nombreTransportista = lector["NombreTransportista"].ToString();
    }
}

```

```

        string TotalProductos = lector["TotalProductos"].ToString();

        // string HoraReg = lector["HoraRegreso"].ToString();
        // string HoraSal = lector["HoraSalida"].ToString();
        string HoraReg =
DateTime.Parse(lector["HoraRegreso"].ToString()).ToString("HH:mm");
        string HoraSal =
DateTime.Parse(lector["HoraSalida"].ToString()).ToString("HH:mm");

        string trasportistayhoras = $"{nombreTransportista} - {HoraSal} -
{HoraReg}";

        // Agrega una nueva fila con los datos formateados
        TablaEntrega.Rows.Add(idEntrega, trasportistayhoras, fechaEntrega, estado,
TotalProductos);

        i++;
    }

    lector.Close(); // Cierra el lector
}
catch (Exception ex)
{
    MessageBox.Show("Error al consultar datos: " + ex.Message);
}
finally
{
    nuevaConexion.Close(); // Cierra la conexión
}
}

public bool MismaRuta(string ruta, string FechaEntrega, SqlConnection conexion)
{
    string query = "SELECT COUNT(1) FROM logistica.Entrega WHERE IdRuta = @IdRuta AND
FechaEntrega = @FechaEntrega";

    using (SqlCommand command = new SqlCommand(query, conexion))
    {
        command.Parameters.AddWithValue("@IdRuta", ruta);
    }
}

```

```

        command.Parameters.AddWithValue("@FechaEntrega", FechaEntrega);

        try
        {
            int count = (int)command.ExecuteScalar();
            return count > 0; // Si el resultado es mayor a 0, significa que ya existe
        }
        catch (SqlException ex)
        {
            MessageBox.Show("Error al verificar al ingresar entrega: " + ex.Message);
            return false;
        }
    }
}

private void InsertaDato()
{
    string IdRuta = ((KeyValuePair<string, string>)comboBoxRuta.SelectedItem).Key;

    try
    {
        nuevaConexion.Open();

        if (MismaRuta(IdRuta, TimeSalida.Text, nuevaConexion))
        {
            nuevaConexion.Close();
            MessageBox.Show("Ya esta registrada Fecha y ruta");
            return;
        }

        // Consulta de inserción
        string insertInfo = "INSERT INTO logistica.Entrega (IdRuta, FechaEntrega, EstadoEntrega, TotalProductos) VALUES ('" + IdRuta + "', '" + TimeSalida.Text + "', '" + "En proceso" + "', '" + 0 + "')";

        SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion);
        cm.ExecuteNonQuery();

        nuevaConexion.Close();

        ActualizarEstadoAutomático();
    }
    catch (SqlException ex)
    {
        // Si ocurre un error SQL (como por capacidad de transporte)
        MessageBox.Show("Capacidad llena del trasporte del transportista: ");
    }
}

```

```

        nuevaConexion.Close();
        return;
    }
}

```

```

public void CargaEstado()
{

```

```

    comboEstado.Items.Clear(); // Limpiar ComboBox

```

```

    comboEstado.Items.Add("No entregado");
    comboEstado.Items.Add("En proceso");
    comboEstado.Items.Add("Entregado");

```

```

}

```

```

private void label3_Click(object sender, EventArgs e)
{
}

```

```

public void CargaRuta()
{

```

```

    try
    {
        nuevaConexion.Open();
        String selectInfo = "SELECT IdRuta, NombreTransportista, HoraRegreso,
Ruta.HoraSalida " +
            "FROM logistica.Ruta " +
            "INNER JOIN logistica.Transportista " +
            "ON Ruta.idTransportista = Transportista.idTransportista";
        SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion);
        SqlDataReader lector = cmd.ExecuteReader();

        comboBoxRuta.Items.Clear(); // Limpiar ComboBox

```

```

while (lector.Read())
{

    string nombreTransportista = lector["NombreTransportista"].ToString();
    // string HoraReg = lector["HoraRegreso"].ToString();
    // string HoraSal = lector["HoraSalida"].ToString();
    string HoraReg =
DateTime.Parse(lector["HoraRegreso"].ToString()).ToString("HH:mm");
    string HoraSal =
DateTime.Parse(lector["HoraSalida"].ToString()).ToString("HH:mm");

    string trasportistayhoras = $"{nombreTransportista} - {HoraSal} -
{HoraReg}";

    comboBoxRuta.Items.Add(new KeyValuePair<string,
string>(lector["IdRuta"].ToString(), trasportistayhoras));
}

comboBoxRuta.DisplayMember = "Value"; // Mostrar el nombre en el ComboBox
comboBoxRuta.ValueMember = "Key"; // El valor será el IdCliente

lector.Close();
}
catch (Exception ex)
{
    MessageBox.Show("Error al cargar la ruta: " + ex.Message);
}
finally
{
    nuevaConexion.Close();
}

}

private void ModificarDato()
{

    if (comboBoxRuta.SelectedItem != null && comboEstado.SelectedItem != null)
    {

```

```

var selectedRuta = (KeyValuePair<string, string>)comboBoxRuta.SelectedItem;
var IdRuta = selectedRuta.Key;

nuevaConexion.Open();

bool modificoRuta = selectedRuta.Value.Equals(Ruta);
bool modificoFecha = Convert.ToDateTime(TimeSalida.Value).ToString("yyyy-MM-
dd").Equals(ent);
bool modificoedo = comboEstado.SelectedItem.Equals(edo);

bool isMod = false;

if (!modificoFecha || !modificoRuta || !modificoedo)
    isMod = true;

if (MismaRuta(IdRuta, TimeSalida.Text, nuevaConexion) && isMod && modificoedo)
{
    nuevaConexion.Close();
    MessageBox.Show("Ya esta registrada Fecha Y Ruta");

    return;
}
/*
!modificoRuta)
        if(MismaRuta(IdRuta, TimeSalida.Text, nuevaConexion) &&
        {
            nuevaConexion.Close();
            MessageBox.Show("Ya esta registrada Ruta");

            return;
        }
*/

```

```

string updateInfo = "UPDATE  logistica.Entrega  SET IdRuta = @IdRuta,

```

```
EstadoEntrega = @EstadoEntrega, FechaEntrega = @FechaEntrega " +  
    "WHERE IdEntrega = @IdEntrega";
```

```
SqlCommand cm = new SqlCommand(updateInfo, nuevaConexion);  
cm.Parameters.AddWithValue("@IdRuta", IdRuta);  
cm.Parameters.AddWithValue("@EstadoEntrega", "En proceso");  
cm.Parameters.AddWithValue("@FechaEntrega", TimeSalida.Value);  
cm.Parameters.AddWithValue("@IdEntrega", IdEntrega);
```

```
cm.ExecuteNonQuery();
```

```
LimpiarCampos();
```

```
nuevaConexion.Close();
```

```
ActualizarEstadoAutomático();
```

```
}
```

```
}
```

```
private void RemoverDato()
```

```
{
```

```
    nuevaConexion.Open();
```

```
    string insertInfo = "DELETE FROM logistica.Entrega WHERE IdEntrega = '" + IdEntrega  
+ "'";
```

```
    SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion);  
    cm.ExecuteNonQuery();
```

```
    nuevaConexion.Close();
```

```
}
```

```
private void botonClickEliminar_Click(object sender, EventArgs e)
```

```
{
```

```

        RemoverDato();
        ConsultaDatos();

        LimpiarCampos();

    }

    private void botonClickModificar_Click(object sender, EventArgs e)
    {

        ModificarDato();
        ConsultaDatos();

        LimpiarCampos();
    }

    private void botonClickInsertar_Click(object sender, EventArgs e)
    {
        InsertaDato();
        ConsultaDatos();

        LimpiarCampos();
    }

    private void label2_Click(object sender, EventArgs e)
    {

    }

    private void TablaEntrega_CellContentClick(object sender, DataGridViewCellEventArgs e)
    {
        if (e.RowIndex >= 0)
        {
            // Obtén la marca seleccionada
            DataGridViewRow filaSeleccionada = TablaEntrega.Rows[e.RowIndex];

            // Obtén el valor de una celda específica (por ejemplo, la primera transporte)
            IdEntrega = filaSeleccionada.Cells[0].Value.ToString();

            string idRutaSeleccionado = filaSeleccionada.Cells[1].Value.ToString();
            Ruta = idRutaSeleccionado;

            foreach (KeyValuePair<string, string> item in comboBoxRuta.Items)
            {
                if (item.Value == idRutaSeleccionado)

```

```

        {
            comboBoxRuta.SelectedItem = item;
            break;
        }
    }
    try
    {
        TimeSalida.Text = filaSeleccionada.Cells[2].Value.ToString();
    }
    catch
    {
        MessageBox.Show("No se puede cargar la Fecha anterior");
    }

    ent = filaSeleccionada.Cells[2].Value.ToString();

    comboEstado.SelectedItem = filaSeleccionada.Cells[3].Value.ToString();
    edo = filaSeleccionada.Cells[3].Value.ToString();

    }
}

```

```

private void LimpiarCampos()
{

    comboEstado.SelectedIndex = -1; ;
    comboBoxRuta.SelectedIndex = -1;

}

```

```

private void ActualizarEstadoAutomático()
{
    try
    {
        nuevaConexion.Open();

        // Consulta para verificar y actualizar el estado
        string query = @"
UPDATE logistica.Entrega
SET EstadoEntrega = CASE
    WHEN FechaEntrega <= CAST(GETDATE() AS DATE) THEN 'Entregado'
    WHEN FechaEntrega > CAST(GETDATE() AS DATE) THEN 'En proceso'

```

```

        ELSE EstadoEntrega
    END
    WHERE EstadoEntrega != 'Entregado'; // Opcional: limitar solo a registros aún no
entregados.

    SqlCommand command = new SqlCommand(query, nuevaConexion);
    int filasAfectadas = command.ExecuteNonQuery();

}
catch (Exception ex)
{
    MessageBox.Show("Error al actualizar los estados: " + ex.Message);
}
finally
{
    nuevaConexion.Close();
}
}
}
}

```

Formulario Detalle Entrega

En la clase DetalleEntrega, se maneja la conexión a la base de datos SQL y se ejecutan diversas consultas para cargar y manipular datos relacionados con las entregas y los almacenamientos. El formulario incluye la carga de datos en un DataGridView, donde se visualizan los detalles de las entregas, como el nombre del producto, fecha de entrega, transportista y ruta. Además, permite seleccionar un almacenamiento y entrega para asociarlos, validando previamente que no exista una relación duplicada entre ambos a través de la función MismaRelacion. Si la relación ya existe, muestra un mensaje de advertencia; si no, permite insertar, modificar o eliminar registros en la base de datos. Los métodos CargaAlmacenamiento, CargaEntrega, y CargaFecha son responsables de llenar los controles de selección como el ComboBox con los valores correspondientes desde la base de datos. El método InsertaDato se utiliza para insertar un nuevo registro en la tabla DetalleEntrega, mientras que ModificarDato actualiza los registros existentes y RemoverDato elimina la relación entre almacenamiento y entrega. Además, las consultas a la base de datos están protegidas por bloques try-catch para manejar excepciones y garantizar el cierre adecuado de las conexiones.

```

public partial class DetalleEntrega : Form
{
    SqlConnection nuevaConexion = Conexion.GetConnection();

    string idEntrega = "", idAlmacenamiento = "";
    string idAlmacenamientoSeleccionado = "";

```

```

DateTime? FechaCompara = null;
public DetalleEntrega()
{
    InitializeComponent();
    CargaEntrega();
    ConsultaDatos();

    TablaDetalleEntrega.Columns[0].Width = 150;
    TablaDetalleEntrega.Columns[1].Width = 160;
    TablaDetalleEntrega.Columns[2].Width = 160;
}

private void ConsultaDatos()
{
    try
    {
        nuevaConexion.Open();
        TablaDetalleEntrega.DataSource = null; // Limpia el DataGridView

        string query = "SELECT AlmacenamientoProducto.IdAlmacenamiento, NombreProducto,
Producto.IdProducto, AlmacenamientoProducto.FechaEntrega, Entrega.IdEntrega, " +
                        "NombreTransportista, Ruta.HoraSalida, DetalleEntrega.FechaReporte " +
                        "FROM logistica.DetalleEntrega " +
                        "INNER JOIN logistica.Entrega ON DetalleEntrega.IdEntrega =
Entrega.IdEntrega " +
                        "INNER JOIN logistica.Ruta ON Entrega.IdRuta = Ruta.IdRuta " +
                        "INNER JOIN logistica.Transportista ON Ruta.IdTransportista =
Transportista.IdTransportista " +
                        "INNER JOIN almacen.AlmacenamientoProducto ON
AlmacenamientoProducto.IdAlmacenamiento = DetalleEntrega.IdAlmacenamiento " +
                        "INNER JOIN almacen.Producto ON AlmacenamientoProducto.IdProducto =
Producto.IdProducto";

        SqlCommand scm = new SqlCommand(query, nuevaConexion);
        SqlDataReader lector = scm.ExecuteReader();

        if (TablaDetalleEntrega.Columns.Count == 0)
        {
            TablaDetalleEntrega.Columns.Add("IdAlmacenamiento", "Almacenamiento");
            TablaDetalleEntrega.Columns.Add("IdEntrega", "Entrega");
            TablaDetalleEntrega.Columns.Add("FechaReporte", "Fecha"); // Nueva columna
        }

        TablaDetalleEntrega.Rows.Clear(); // Limpia las filas del DataGridView

        while (lector.Read())
        {
            string IdAlmacenamiento = lector["IdAlmacenamiento"].ToString();

```

```

        string NombreProducto = lector["NombreProducto"].ToString();
        string IdProducto = lector["IdProducto"].ToString();
        string FechaEntrega =
DateTime.Parse(lector["FechaEntrega"].ToString()).ToString("yyyy-MM-dd");
        string IdEntrega = lector["IdEntrega"].ToString();
        string nombreTransportista = lector["NombreTransportista"].ToString();
        string horaSalida =
DateTime.Parse(lector["HoraSalida"].ToString()).ToString("HH:mm");
        string fechaReporte =
DateTime.Parse(lector["FechaReporte"].ToString()).ToString("yyyy-MM-dd HH:mm");

        string infoAlmacenamiento = $"{IdAlmacenamiento} - {IdProducto} -
{NombreProducto} - {FechaEntrega}";
        string infoEntrega = $"{IdEntrega} - {nombreTransportista} ({horaSalida})";

        // Agrega los datos al DataGridView
        TablaDetalleEntrega.Rows.Add(infoAlmacenamiento, infoEntrega, fechaReporte);
    }

    lector.Close();
    nuevaConexion.Close();
}
catch (Exception ex)
{
    MessageBox.Show("Error al cargar los datos: " + ex.Message);
}
}

private void CargaAlmacenamiento()
{
    if (FechaCompara != null)
    {
        try
        {
            nuevaConexion.Open();

            // Consulta para obtener los datos
            string selectInfo = "SELECT AlmacenamientoProducto.IdAlmacenamiento,
AlmacenamientoProducto.IdProducto, Producto.NombreProducto, FechaEntrega " +
                                "FROM almacen.AlmacenamientoProducto " +
                                "INNER JOIN almacen.Producto " +
                                "ON AlmacenamientoProducto.IdProducto = Producto.IdProducto "
+
                                "WHERE FechaSalida = @FechaEntrega";

            SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion);
            cmd.Parameters.AddWithValue("@FechaEntrega", FechaCompara);

            SqlDataReader lector = cmd.ExecuteReader();

```

```

// Almacenar temporalmente los datos del lector
List<KeyValuePair<string, string>> datos = new List<KeyValuePair<string,
string>>();

while (lector.Read())
{
    string IdAlmacenamiento = lector["IdAlmacenamiento"].ToString();
    string NombreProducto = lector["NombreProducto"].ToString();
    string IdProducto = lector["IdProducto"].ToString();
    string FechaEntrega =
DateTime.Parse(lector["FechaEntrega"].ToString()).ToString("yyyy-MM-dd");

    string infoEntrega = $"{IdAlmacenamiento} - {IdProducto} - {NombreProducto} -
{FechaEntrega}";
    datos.Add(new KeyValuePair<string, string>(IdAlmacenamiento, infoEntrega));
}

lector.Close(); // Cerrar el lector antes de realizar otras operaciones

// Limpiar ComboBox antes de llenarlo
comboBoxAlmacenamiento.Items.Clear();

foreach (var dato in datos)
{

    if (dato.Value == idAlmacenamientoSeleccionado)
    {

        idAlmacenamiento = dato.Key;
        break;
    }
}

// Procesar los datos almacenados
foreach (var dato in datos)
{
    string idAlmacenamiento2 = dato.Key;
    string infoEntrega = dato.Value;

    // Crear una nueva conexión para llamar a MismaRelacion
    using (SqlConnection nuevaConexion2 = new
SqlConnection(nuevaConexion.ConnectionString))
    {
        nuevaConexion2.Open();

        if (!MismaRelacion(idAlmacenamiento2, ((KeyValuePair<string,
string>)comboBoxEntrega.SelectedItem).Key, nuevaConexion2))
        {

```

```

        comboBoxAlmacenamiento.Items.Add(new KeyValuePair<string,
string>(idAlmacenamiento2, infoEntrega));
    }
    nuevaConexion2.Close();
}

// Configurar cómo se mostrarán los datos en el ComboBox
comboBoxAlmacenamiento.DisplayMember = "Value"; // Mostrar el nombre en el
ComboBox
comboBoxAlmacenamiento.ValueMember = "Key"; // El valor será el
IdAlmacenamiento
}
catch (Exception ex)
{
    MessageBox.Show("Error al cargar la ruta: " + ex.Message);
}
finally
{
    nuevaConexion.Close(); // Asegurarse de cerrar la conexión principal
}
}
}

```

```

private void CargaEntrega()
{
    try
    {
        nuevaConexion.Open();

        String selectInfo = "SELECT IdEntrega, Transportista.NombreTransportista,
Ruta.HoraSalida " +
        "FROM logistica.Entrega " +
        "INNER JOIN logistica.Ruta " +
        "ON Entrega.IdRuta = Ruta.IdRuta " +
        "INNER JOIN logistica.Transportista ON Ruta.IdTransportista =
Transportista.IdTransportista ";

        SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion);
        SqlDataReader lector = cmd.ExecuteReader();

        comboBoxEntrega.Items.Clear(); // Limpiar ComboBox

        while (lector.Read())
        {

```

```

        string IdEntrega = lector["IdEntrega"].ToString();
        string nombreTransportista = lector["NombreTransportista"].ToString();
        string horaSalida =
DateTime.Parse(lector["HoraSalida"].ToString()).ToString("HH:mm");

```

```

        string infoAlmacenamiento = $"{IdEntrega} - {nombreTransportista}
({horaSalida})";

```

```

        comboBoxEntrega.Items.Add(new KeyValuePair<string,
string>(lector["IdEntrega"].ToString(), infoAlmacenamiento));
    }

```

```

        comboBoxEntrega.DisplayMember = "Value"; // Mostrar el nombre en el ComboBox
comboBoxEntrega.ValueMember = "Key"; // El valor será el IdCliente

```

```

        lector.Close();
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error al cargar la ruta: " + ex.Message);
    }
    finally
    {
        nuevaConexion.Close();
    }
}

```

```

private void CargaFecha()
{
    try
    {
        string IdEntrega = ((KeyValuePair<string, string>)comboBoxEntrega.SelectedItem).Key;

        nuevaConexion.Open();

        String selectInfo = "SELECT FechaEntrega FROM logistica.Entrega WHERE IdEntrega =
@IdEntrega ";

        SqlCommand cmd = new SqlCommand(selectInfo, nuevaConexion);
        cmd.Parameters.AddWithValue("@IdEntrega", IdEntrega);
    }
}

```

```

        SqlDataReader lector = cmd.ExecuteReader();

        while (lector.Read())
        {

            FechaCompara = DateTime.Parse(lector["FechaEntrega"].ToString());

        }

        lector.Close();
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error al cargar la fecha: " + ex.Message);
    }
    finally
    {
        nuevaConexion.Close();
    }
}

private bool MismaRelacion(string IdAlmacenamiento, string IdEntrega, SqlConnection conexion)
{
    string query = "SELECT COUNT(1) FROM logistica.DetalleEntrega WHERE IdAlmacenamiento = @IdAlmacenamiento AND IdEntrega = @IdEntrega";

    using (SqlCommand command = new SqlCommand(query, conexion))
    {
        command.Parameters.AddWithValue("@IdEntrega", IdEntrega);
        command.Parameters.AddWithValue("@IdAlmacenamiento", IdAlmacenamiento);

        try
        {
            int count = (int)command.ExecuteScalar();
            return count > 0; // Si el resultado es mayor a 0, significa que ya existe
        }
        catch (SqlException ex)
        {
            MessageBox.Show("Error al verificar al ingresar detalleentrega: " + ex.Message);
            return false;
        }
    }
}

```

```

    }
}

private void InsertaDato()
{
    string IdEntrega = ((KeyValuePair<string, string>)comboBoxEntrega.SelectedItem).Key;
    string IdAlmacenamiento = ((KeyValuePair<string,
string>)comboBoxAlmacenamiento.SelectedItem).Key;

    nuevaConexion.Open();

    if (MismaRelacion(IdAlmacenamiento, IdEntrega, nuevaConexion))
    {
        nuevaConexion.Close();
        MessageBox.Show("Ya esta registrada El almacenamiento y entrega");

        return;
    }

    DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss");

    string insertInfo = "INSERT INTO logistica.DetalleEntrega (IdAlmacenamiento, IdEntrega,
FechaReporte) " +
        "VALUES (@IdAlmacenamiento, @IdEntrega, @FechaReporte)";

    SqlCommand cm = new SqlCommand(insertInfo, nuevaConexion);
    cm.Parameters.AddWithValue("@IdAlmacenamiento", IdAlmacenamiento);
    cm.Parameters.AddWithValue("@IdEntrega", IdEntrega);
    cm.Parameters.AddWithValue("@FechaReporte", DateTime.Now);
    cm.ExecuteNonQuery();

    nuevaConexion.Close();
}

private void ModificarDato()
{
    if (comboBoxAlmacenamiento.SelectedItem != null && comboBoxEntrega.SelectedItem != null)
    {
        string IdAlmacenamiento = ((KeyValuePair<string,
string>)comboBoxAlmacenamiento.SelectedItem).Key;
        string IdEntrega = ((KeyValuePair<string, string>)comboBoxEntrega.SelectedItem).Key;

```

```

        bool modificoAlmacenamiento = IdAlmacenamiento.Equals(idAlmacenamiento);
        bool modificoEntrega = IdEntrega.Equals(idEntrega);

        nuevaConexion.Open();

        if (MismaRelacion(IdAlmacenamiento, IdEntrega, nuevaConexion) &&
            (!modificoAlmacenamiento || !modificoEntrega))
        {
            nuevaConexion.Close();
            MessageBox.Show("Ya está registrada la relación entre el almacenamiento y la
entrega.");
            return;
        }

        try
        {
            // Obtén la fecha y hora actual
            string fechaActual = DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss");

            // Actualiza los datos en la base de datos
            string updateQuery = "UPDATE logistica.DetalleEntrega " +
                                "SET IdEntrega = @IdEntrega, IdAlmacenamiento =
@IdAlmacenamiento, FechaReporte = @FechaReporte " +
                                "WHERE IdEntrega = @IdEntregaAnterior AND IdAlmacenamiento =
@IdAlmacenamientoAnterior";

            SqlCommand cm = new SqlCommand(updateQuery, nuevaConexion);
            cm.Parameters.AddWithValue("@IdEntrega", IdEntrega);
            cm.Parameters.AddWithValue("@IdAlmacenamiento", IdAlmacenamiento);
            cm.Parameters.AddWithValue("@FechaReporte", DateTime.Now);
            cm.Parameters.AddWithValue("@IdEntregaAnterior", idEntrega);
            cm.Parameters.AddWithValue("@IdAlmacenamientoAnterior", idAlmacenamiento);

            cm.ExecuteNonQuery();

            MessageBox.Show("Datos modificados correctamente.");
        }
        catch (Exception ex)
        {
            MessageBox.Show("Error al modificar los datos: " + ex.Message);
        }
        finally
        {
            nuevaConexion.Close();
        }
    }
    else
    {

```

```

        MessageBox.Show("Por favor, selecciona un almacenamiento y una entrega para
modificar.");
    }
}

private void RemoverDato()
{
    string IdAlmacenamiento = idAlmacenamiento;
    string IdEntrega = idEntrega;

    try
    {
        nuevaConexion.Open();

        // Elimina primero los registros relacionados en DetalleEntrega
        string deleteDetalleEntrega = "DELETE FROM logistica.DetalleEntrega WHERE
IdAlmacenamiento = @IdAlmacenamiento AND IdEntrega = @IdEntrega";
        SqlCommand cmdDetalleEntrega = new SqlCommand(deleteDetalleEntrega, nuevaConexion);
        cmdDetalleEntrega.Parameters.AddWithValue("@IdAlmacenamiento", IdAlmacenamiento);
        cmdDetalleEntrega.Parameters.AddWithValue("@IdEntrega", IdEntrega);
        cmdDetalleEntrega.ExecuteNonQuery();

    }
    catch (SqlException ex)
    {
        MessageBox.Show("Error al eliminar el registro: " + ex.Message);
    }
    finally
    {
        nuevaConexion.Close();
    }
}

private void botonClickEliminar_Click(object sender, EventArgs e)
{
    RemoverDato();
    ConsultaDatos();

    LimpiarCampos();

}

private void botonClickModificar_Click(object sender, EventArgs e)
{
    ModificarDato();
}

```

```

        ConsultaDatos();

        LimpiarCampos();
    }

    private void botonClickInsertar_Click(object sender, EventArgs e)
    {
        InsertaDato();
        ConsultaDatos();

        LimpiarCampos();
    }

    private void LimpiarCampos()
    {

        comboBoxEntrega.SelectedIndex = -1; ;
        comboBoxAlmacenamiento.SelectedIndex = -1;
        comboBoxAlmacenamiento.Items.Clear();

    }

    private void DetalleEntrega_Load(object sender, EventArgs e)
    {

    }

    private void comboBoxEntrega_SelectedIndexChanged(object sender, EventArgs e)
    {
        if (comboBoxEntrega.SelectedItem != null)
        {
            comboBoxAlmacenamiento.SelectedIndex = -1;

            CargaFecha();
            CargaAlmacenamiento();
        }
    }

    private void TablaDetalleEntrega_CellContentClick(object sender, DataGridViewCellEventArgs e)
    {
        if (e.RowIndex >= 0)
        {
            // Obtén la marca seleccionada
            DataGridViewRow filaSeleccionada = TablaDetalleEntrega.Rows[e.RowIndex];

```

```
// Obtén el valor de una celda específica (por ejemplo, la primera transporte)
```

```
idAlmacenamientoSeleccionado = filaSeleccionada.Cells[0].Value.ToString();
```

```
string idEntregaSeleccionado = filaSeleccionada.Cells[1].Value.ToString();
```

```
foreach (KeyValuePair<string, string> item in comboBoxEntrega.Items)
{
```

```
    if (item.Value == idEntregaSeleccionado)
```

```
    {
```

```
        idEntrega = item.Key;
```

```
        comboBoxEntrega.SelectedItem = item;
```

```
        break;
```

```
    }
```

```
}
```

```
}
```

```
}
```

```
}
```